

DV Agent Harness L5 實務上機 Lab 手冊

DV Agent Harness L5 -- 實務上機 Lab 手冊

這份文件是 DV Agent Harness L5 完整參考手冊(docs/DV_Agent_Harness_L5_Detailed_User_Guide_TC_2026-09-06.md)的實務上機companion——內容全部是可以在真實終端機上一步一步照著做的 Lab,每個 Lab 都包含學習目標、前置準備、實測過的步驟與真實指令輸出、驗證清單,以及常見問題。目的是讓使用者不只讀懂這套系統的設計,還能親手在自己的機器上動手操作、看到真實輸出、建立起「以當下證據為準,不以文件或記憶為準」的操作直覺。建議搭配完整參考手冊交叉閱讀:遇到本手冊沒展開的細節,可回到參考手冊查對應章節;若本手冊的敘述與你眼前的真實指令輸出不一致,一律以你自己跑出來的真實輸出為準。

文件日期:2026-09-07

目錄

- [Lab 0: 環境檢查與 CLI 初體驗](#)
 - [Lab 1: 建立一個新專案並執行 Intake](#)
 - [Lab 2: DUT RTL / Register / PHY Discovery 實作](#)
 - [Lab 3: VIP Discovery 與 API 可驗證性檢查](#)
 - [Lab 4: 產生一個 Subsystem-Mode UVM 驗證環境](#)
 - [Lab 5: AMBA 匯流排拓樸與交易分析上機練習](#)
 - [Lab 6: Spec-to-vPlan 與需求可測試性檢查](#)
 - [Lab 7: 模擬結果解析與 LSF Regression 概念\(無真實伺服器時的替代路徑\)](#)
 - [Lab 8: Coverage 結案、Waiver 與 Golden Scenario 上機練習](#)
 - [Lab 9: Loop Engineering、AI 推理與人機互動\(question_queue\)實作](#)
 - [Lab 10: Dashboard/GUI 啟動與操作導覽](#)
 - [Lab 11: 疑難排解與下一步學習資源](#)
-

Lab 0: 環境檢查與 CLI 初體驗

學習目標

- 確認本機 Python 環境能夠正確匯入 `dv_harness` 套件,並理解為何在本沙盒環境中要用 `python -m dv_harness.cli` 而不是直接打 `dv-harness`。
- 實際執行 `python -m dv_harness.cli --help`,從真實輸出中讀懂這個擁有 200 多個動詞的 CLI 是怎麼被劃分成幾大主題群組的。
- 學會兩個唯讀、安全、不會動到專案狀態的「健康檢查」指令: `status` 與 `platform-startup-readiness`,並看懂它們回報的真實欄位。
- 建立起「CLAUDE.md 只是指引、不是證據」的 Evidence Truth Rule 心態,學會用指令的即時輸出而不是記憶去確認系統行為。

前置準備

- 已在本機 clone 好 `DV_Agent_Harness_L5` 專案,本課以 `D:\DV\Task\DV_Agent_Harness_L5\v50` 為工作目錄。
- 已安裝可用的 Python 直譯器,且該目錄下能找到 `dv_harness` 這個 package(即 `dv_harness/__init__.py` 存在)。
- 本課全程只執行唯讀指令,不需要連線到真實 LSF 農場或 license server,也不會修改任何專案檔案 —— 適合完全沒有農場帳號的新人在自己筆電上做。

步驟

Step 1 -- 確認 `dv_harness` 可以被匯入

```
cd D:/DV/Task/DV_Agent_Harness_L5/v50
python -c "import dv_harness; print(dv_harness.__file__)"
```

這一步只是最基本的健康檢查:確認你現在所在的目錄底下的 `dv_harness` 套件可以被 Python 找到並成功載入。實際執行後會看到:

```
D:\DV\Task\DV_Agent_Harness_L5\v50\dv_harness\__init__.py
```

如果這裡噴 `ModuleNotFoundError`,代表你不在專案根目錄下執行,或是 `PYTHONPATH` / 虛擬環境不對 —— 先解決這個,後面所有步驟都不用做了。

Step 2 -- 看 CLI 的真實頂層說明

```
python -m dv_harness.cli --help
```

這一步印出 `dv-harness` 这支 CLI 目前真正註冊的完整動詞清單與每個動詞的一行說明(argparse 的 subparsers 說明區塊),第一行還會告訴你目前是哪個版本:

```
DV Agent Harness Edition v15
```

實際跑一次你會看到 `{start,run-stage,status,advance,stats,set-stage,mark,explain,pause,resume,...}` 這種一長串、超過 200 個動詞的清單。第一次看到會很嚇人,但如果照著真實輸出的動詞名字本身去分類,可以看出幾

個清楚的主題群:

- 人工控制平面 / 執行控制: `status`、`pause`、`resume`、`takeover`、`release-takeover`、`redirect`、`approve`、`evidence`、`checklist`、`correct`、`constraint`、`cosign`、`explain`、`advance`、`mark`、`set-stage`、`run-stage`。這一群幾乎都對應 CLAUDE.md 講的 Human Control Plane(人類可以暫停/接管/核准某個 stage)。
- LSF / 農場與本地排程: `lsf`、`lsf-submit`、`preflight`、`lsf-kill`、`lsf-reconcile`、`lsf-watch-start/stop/status`、`pueue`、`lsf-auto-kill-scan`。這一群都跟真正送 job 到 LSF 農場、或用 pueue 做本機任務鍵有關。
- 治理 / 稽核 / 風險: `audit`、`git-guard`、`blast-radius`、`self-audit`、`exemptions`、`supply-chain`、`dependency-qualification`。
- 知識與記憶: `knowledge`、`memory`、`memory-store`、`cross-project`、`user-info`、`save-session`、`restore-session`。
- 系統級 SYS-1..39 分析鏈: `subsystem-discovery` → `subsystem-analysis` → `system-resource-inventory` → `system-integration-plan` → `system-command-plan` → `system-scheduling-plan` → `system-topology-analysis` → `system-phase1-report`, 再加上 `system-smoke-proof`, 這條是一整串照順序執行的多子系統整合分析管線。
- 協定/介面 IR 建模(名字常以 `-ir` 結尾或以 `amba-` 開頭): `amba-functional-coverage-ir`、`amba-readiness-gates`、`arbitration-policy-ir`、`coherency-capability-ir`、`backpressure-model`、`fabric-progress`、`programming-sequence-ir`、`protocol-capability`、`protocol-compliance-oracle` 等等, 這一群是把 spec/RTL 證據轉成結構化 IR、再做規則判斷, 幾乎都聲明「Reads only」。
- 平台自我健檢: `platform-startup-readiness`、`platform-health`、`self-tune`、`golden-flow-readiness`、`generation-readiness`。

這種「先跑 `--help` 再照真實動詞名分組」的做法, 就是本課要教會你的方法 —— 千萬不要靠記憶或別的文件去猜某個動詞存不存在, 一律以這次跑出來的清單為準。

Step 3 -- 用 `status --help` 看子檢視有哪些

```
python -m dv_harness.cli status --help
```

真實輸出:

```
usage: dv-harness status [-h] [--json]
                        [{full,blockers,jobs,coverage,agents,system,evidence,signoff}]

positional arguments:
  {full,blockers,jobs,coverage,agents,system,evidence,signoff}
                        Optional HarnessStatusIR subview; omit for the
                        unchanged bare summary.

options:
  -h, --help            show this help message and exit
  --json                With a subview: print the selected fields as JSON
                        instead of plain text.
```

也就是說 `status` 這一個動詞本身還細分了 8 種子檢視 (`full`、`blockers`、`jobs`、`coverage`、`agents`、`system`、`evidence`、`signoff`), 不加參數就是「不變的既有摘要」。

Step 4 -- 跑一次不帶參數的 `status`, 看目前 ENV_CHECK 這個 stage 的真實狀態

```
python -m dv_harness.cli status
```

在這個 v50 專案上實際執行,會拿到類似這樣的真實 JSON(節錄):

```
{
  "project": "DV_Workflow_DE_DV_Agent_Harness_Edition_v15",
  "scope": "unknown",
  "current_stage": "ENV_CHECK",
  "active_stages": [],
  "effective_active_stages": ["ENV_CHECK"],
  "overall_status": "PASS",
  "paused": false,
  "takeover_active": false,
  "operation_mode": "NORMAL",
  "degraded": false,
  "degraded_probe_transport": {
    "requested": "auto",
    "resolved": "none",
    "available": false,
    "reason": "auto: no usable probe transport here -- relay: RELAY_NOT_READY; local:
      LOCAL_COMMANDS_MISSING: lmutil, bqueues. ..."
  },
  "dry_run_mode": false
}
```

注意 `current_stage` 正好就是 `"ENV_CHECK"` —— 這代表這個專案目前的引擎狀態就停在「環境檢查」這個 stage,和你本課要學的主題完全對上。同時你也會看到 `degraded_probe_transport.resolved` 是 `"none"`,原因寫得很清楚:本機找不到 `lmutil`、`bqueues` 這些真實的 license/排程探測工具 —— 這是本沙盒訓練環境沒有真正連上 `license server` 與 `LSF` 農場的正常現象,不代表指令壞掉。

Step 5 -- 跑一次真正的、唯讀的平台起始健檢: `platform-startup-readiness`

```
python -m dv_harness.cli platform-startup-readiness
```

這個指令檢查的是「harness 自己在任何 stage 都還沒跑之前,設定是否正確、宣告的 Python 依賴是否真的裝了、`config.json` 是否合法」,完全不碰任何 DUT/RTL 證據。在本機實際跑出來的真實結果:

```
DV Agent Harness L5 -- platform startup readiness (D:\DV\Task\DV_Agent_Harness_L5\v50)
OVERALL: BLOCKED
```

SUBSYSTEM	STATUS	REASON
harness_configuration	READY	CONFIG_JSON_VALID
python_dependencies	BLOCKED	DECLARED_DEPENDENCY_NOT_INSTALLED 2 declared Python dependency(ies) not installed for this interpreter: ['claude-code-sdk', 'setuptools']
agent_adapter	READY	AGENT_ADAPTER_COMMAND_RESOLVABLE config.claude.command='claude' resolves to 'C:\Users\...\claude.CMD'
execution_preflight_configuration	READY	PREFLIGHT_CONFIG_SHAPE_VALID
project_state_directory	READY	STATE_DIRECTORY_WRITABLE

這裡看到的 `BLOCKED` 是一個**真實、誠實回報**的結果,不是你操作錯誤——這台訓練機的 Python 直譯器確實沒裝 `claude-code-sdk` 和 `setuptools` 這兩個 `pyproject.toml` 宣告的依賴。這正是這個指令存在的意義:在任何 stage 真正跑之前,就先誠實告訴你「這裡有洞,不要假裝一切 `READY`」。

驗證

完成本課後,請對照確認以下幾點:

- ☐ `python -c "import dv_harness; print(dv_harness.__file__)"` 能印出 `...\v50\dv_harness\__init__.py` 而不報錯。
- ☐ `python -m dv_harness.cli --help` 能列出以 `DV Agent Harness Edition v15` 開頭、包含 `status`、`lsf-submit`、`subsystem-discovery`、`platform-startup-readiness` 等動詞在內的完整清單,並且你能說出至少 3 個你自己歸納出的主題群組(例如「人工控制平面」「LSF 農場」「系統級 SYS 分析鏈」)。
- ☐ `python -m dv_harness.cli status` 能回傳一段 JSON,且 `current_stage` 欄位有值(本次應為 `"ENV_CHECK"`)。
- ☐ `python -m dv_harness.cli platform-startup-readiness` 能跑完並印出一個 `OVERALL:` 判定(`READY` 或 `BLOCKED` 皆屬正常,取決於你機器實際裝了哪些依賴)。

常見問題

1. 直接打 `dv-harness --help` 出現 `command not found`,是不是壞了? 不是。`pyproject.toml` 裡確實宣告了 `dv-harness = "dv_harness.cli:main"` 這個 console script 進入點,但只有在你對這個專案做過 `pip install -e .` (或等效安裝)之後, `dv-harness` 這個可執行檔才會出現在 `PATH` 上。在還沒安裝、或在乾淨的訓練沙盒裡,一律改用 `python -m dv_harness.cli <verb> ...`,行為完全等價,這也是本課全程使用的形式。
 2. `platform-startup-readiness` 回報 `python_dependencies BLOCKED`,要不要現在去 `pip install`? 本課的目的是「看懂真實輸出」,不是修環境,所以不需要在這裡動手安裝。真正動手修復依賴是後續課程的範圍;此處請把這個 `BLOCKED` 當成活教材:它示範了這個 harness 對「未驗證的事」寧可老實回報 `BLOCKED / NOT_AVAILABLE`,也不會假裝 `READY`,這正是專案 `CLAUDE.md` 開頭 Evidence Truth Rule(「`CLAUDE.md` 是指引不是驗證證據,任何工程結論都要用當下的真實證據驗證,證據與文件衝突時以證據為準」)在 CLI 層級的具體體現。
 3. `status` 輸出裡 `degraded_probe_transport.resolved` 是 `"none"` 且列出 `lmutil`、`bqueues` 缺失,代表 `license/排程壞了嗎?` 在沒有真正連上 license server 與 LSF farm 的訓練環境裡,這是預期行為,不是錯誤: `-degradation-transport auto` (預設值)只有在真的偵測到可用的 relay 或本機 `lmutil / bqueues` 時才會啟用資源探測,寧可什麼都不探測,也不要把一個「根本找不到的指令」誤讀成「license 滿了」或「農場卡住了」。若你之後真的連到有 LSF/license 的機器,可用 `--degradation-transport local` 或 `remote_relay` 明確指定要用哪種探測方式,再用 `status` 觀察 `degraded_probe_transport` 欄位的變化。
-

Lab 1: 建立一個新專案並執行 Intake

學習目標

- 學會在一個全新的、空白的專案目錄上初始化 DV Agent Harness 的 `.dv-harness/` 狀態,並理解 `status` 指令如何在背後完成這件事。
- 動手操作 `intake_baseline.py` 的完整 CLI 流程(`fields -> baseline -> freeze -> list -> status`),理解「凍結」(freeze)一份 intake baseline 的真實語意與檔案落點。
- 實際啟動 GUI-01 互動式 Intake Wizard(`gui_intake_wizard.py`)本機伺服器,並用真實的 HTTP 請求檢視它回傳的 14 個步驟與 grounded 狀態。
- 學會用 `intake-modes` 與 `intake-events` 兩個唯讀指令查詢 harness 內建的 intake 模式 (FAST/STANDARD/STRICT/SIGNOFF)與事件字彙表。

前置準備

- 已經可以在 `D:\DV\Task\DV_Agent_Harness_L5\v50` 目錄下執行 `python -m dv_harness.cli --help` 並看到完整的 verb 清單(如果看不到,代表 Python 環境或 `dv_harness` 套件路徑有問題,先解決這個再繼續)。
- 絕對不要在 `v50` 這個正式專案目錄本身上執行本課的指令——這些指令會寫入 `.dv-harness/` 狀態。本課一律在一個新建立的「scratch」目錄上操作,例如
`D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project`。
- 手邊要有一個可以寫入暫存檔案的資料夾,用來放 `--facts` 需要的 JSON 檔(見步驟 3)。
- 具備基本的終端機操作能力(這裡示範用 Git Bash/PowerShell 皆可,指令本身不變)。

步驟

步驟 1:建立全新的 scratch 專案目錄,並用 `status` 觸發初始化

在 `v50` 目錄下執行(注意 `--project-root` 指向一個尚未存在 `.dv-harness/` 的全新資料夾):

```
mkdir -p "D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project"
cd "D:\DV\Task\DV_Agent_Harness_L5\v50"
python -m dv_harness.cli --project-root
"D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project" status
```

這一步做的事: `status` 是唯讀查詢,但當它發現 `--project-root` 底下還沒有 `.dv-harness/` 狀態時,會自動幫你搭整套骨架建起來(`state.json` 、 `control.json` 、 `config.json` 、 `events.jsonl` 、 `agents/` 、 `blackboard/` 、 `plans/` 、 `react/` 、 `telemetry/`)。

實際輸出(本課實測,完整節錄前段):

```
{
  "project": "my_new_project",
  "scope": "unknown",
  "current_stage": "ENV_CHECK",
  "active_stages": [],
  "effective_active_stages": ["ENV_CHECK"],
  "overall_status": "NOT_STARTED",
  ...
  "degraded_probe_transport": {
```

```

    "requested": "auto",
    "resolved": "none",
    "available": false,
    "reason": "auto: no usable probe transport here -- relay: RELAY_NOT_READY; local:
        LOCAL_COMMANDS_MISSING: lmutil, bqueues. ..."
  },
  ...
}

```

注意 `current_stage` 一開始就是 `ENV_CHECK`, `degraded_probe_transport.resolved` 是 `"none"` (因為這台訓練用機器上沒有 `lmutil` / `bqueues` 這些真正的 LSF/授權工具——這是本課環境的真實限制, harness 誠實回報而不是假裝有 farm 連線)。用下面指令確認骨架真的建好了:

```
find "D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project" -maxdepth 4
```

你會看到 `.dv-harness/agents`、`.dv-harness/blackboard`、`.dv-harness/config.json`、`.dv-harness/control.json`、`.dv-harness/events.jsonl`、`.dv-harness/state.json` 等實體檔案。

步驟 2: 查詢 intake baseline 要抓哪十二個欄位

```
python -m dv_harness.cli intake-baseline --root
"D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project" fields
```

這一步做的事: 列出 `intake_baseline.py` 認得的十二個 intake 欄位名稱、每個欄位可能的狀態值 (`CAPTURED` / `NOT_AVAILABLE`)、以及 freeze 記錄可能的三種驗證狀態 (`VALID` / `UNKNOWN` / `INVALIDATED`)。

實際輸出:

```

{
  "intake_fields": [
    "dut_top_boundary", "dut_sha", "tb_sha", "source_file_hashes",
    "vip_declaration", "bind_topology_hash", "reference_uvm_hash",
    "de_command_txt_hash", "known_test_list",
    "unresolved_critical_unknowns_count", "unresolved_conflicts_count",
    "recorded_user_decisions_count"
  ],
  "field_statuses": ["CAPTURED", "NOT_AVAILABLE"],
  "freeze_statuses": ["VALID", "UNKNOWN", "INVALIDATED"]
}

```

再試著在還沒有任何 freeze 記錄的情況下查 `status`, 你會得到一個「被拒絕」的誠實回應, 而不是假裝一個空的通過結果:

```
python -m dv_harness.cli intake-baseline --root
"D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project" status
```

```

{
  "status": "REFUSED",
  "reason": "CURRENT_FACTS_REQUIRED",
  "detail": "comparing a frozen baseline against nothing would misreport every captured field as
    evidence that disappeared"
}

```

這正是 Evidence Truth Rule 精神的具體實作:「查不到」永遠不能被包裝成「通過」。

步驟 3:準備一份真實的 facts JSON,呼叫 `baseline` 產生報告

`--facts` 吃的是檔案路徑,不是行內 JSON 字串(這是本課實測發現、務必注意的真實語法)。先寫一個 `facts.json`:

```
{
  "dut_top_boundary": {"status": "CAPTURED", "digest": "abc123"},
  "dut_sha": {"status": "CAPTURED", "digest": "sha1"}
}
```

存到 `D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\facts.json` ,然後執行:

```
python -m dv_harness.cli intake-baseline \
--root "D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project" \
--facts "D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\facts.json" \
baseline
```

這一步做的事:對你提供的原始 facts 逐欄位計算 digest(用 `source_identity.aggregate_source_id()` 統一雜湊,不是自己另發明一套雜湊法),回報每個欄位是 `CAPTURED` 還是 `NOT_AVAILABLE` ,但還沒有寫入任何檔案——這只是預覽,是否要真的凍結由你決定。

實際輸出(節錄):

```
{
  "schema_version": "1.0",
  "captured_at": "2026-09-07T04:53:51.777799+00:00",
  "fields": {
    "dut_top_boundary": {
      "field": "dut_top_boundary", "status": "CAPTURED",
      "reason": "DECLARED_FACT_HASHED",
      "digest": "44101f9a8a715bdb5831598edeedffc70b8eee626eea8e9d6d49bbe45dca7ff5"
    },
    "dut_sha": {
      "field": "dut_sha", "status": "CAPTURED",
      "reason": "MULTI_FILE_CONTENT_AGGREGATED", ...
    },
    "tb_sha": {"field": "tb_sha", "status": "NOT_AVAILABLE", "reason": "NO_FACT_SUPPLIED", ...},
    ...
  }
}
```

其餘沒提供的欄位一律誠實回報 `NOT_AVAILABLE` / `NO_FACT_SUPPLIED` (或計數欄位的 `NO_COUNT_SUPPLIED`),絕不會用猜測填補。

步驟 4:真正凍結(freeze)這份 baseline

`freeze` 必須指定 `--frozen-by` (不具名的 `freeze` 會被拒絕,這是刻意設計):

```
python -m dv_harness.cli intake-baseline \
--root "D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project" \
--facts "D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\facts.json" \
freeze
```

```
--frozen-by "lab-student" \  
--note "Lab 1 first freeze" \  
freeze
```

這一步做的事:把步驟 3 的 baseline 寫成一份不可變的 JSON 記錄到 `.dv-harness/intake/baselines/<freeze_id>.json`,只寫這一個檔案,不碰 `state.json`、不碰 `events.jsonl`,也不觸發任何 stage gate。

實際輸出開頭:

```
{  
  "schema_version": "1.0",  
  "freeze_id": "e59534e2bb781f2e",  
  "frozen_at": "2026-09-07T04:54:10.539275+00:00",  
  "frozen_by": "lab-student",  
  "note": "Lab 1 first freeze",  
  "project_root": "D:\\DV\\Task\\DV_Agent_Harness_L5\\v50_lab_scratch\\my_new_project",  
  "baseline": { ... }  
}
```

確認實體檔案真的落地了:

```
find "D:\\DV\\Task\\DV_Agent_Harness_L5\\v50_lab_scratch\\my_new_project\\.dv-harness\\intake"
```

會看到 `.dv-harness/intake/baselines/e59534e2bb781f2e.json`。

步驟 5:用 `list` 與 `status` 驗證凍結記錄

```
python -m dv_harness.cli intake-baseline --root  
      "D:\\DV\\Task\\DV_Agent_Harness_L5\\v50_lab_scratch\\my_new_project" list  
python -m dv_harness.cli intake-baseline --root  
      "D:\\DV\\Task\\DV_Agent_Harness_L5\\v50_lab_scratch\\my_new_project" --facts  
      "D:\\DV\\Task\\DV_Agent_Harness_L5\\v50_lab_scratch\\facts.json" status
```

`list` 回傳一個陣列,列出所有 freeze 記錄的 id/時間/frozen_by; `status` (這次帶著同一份 `facts.json`) 重新對比「現在的事實」跟「當初凍結的事實」,因為 `facts` 沒變,你應該看到頂層 `"status": "VALID"`、`"counts": {"VALID": 1, "UNKNOWN": 0, "INVALIDATED": 0}`,以及 `"record_integrity": "RECORD_UNCHANGED"`。這正是 `evaluate_intake_freeze_invalidation()` 的「每次都重新推導、絕不相信儲存的結論」設計。

步驟 6:啟動 GUI-01 互動式 Intake Wizard,實際打 API

```
python -m dv_harness.gui_intake_wizard \  
  --project-root "D:\\DV\\Task\\DV_Agent_Harness_L5\\v50_lab_scratch\\my_new_project" \  
  --host 127.0.0.1 --port 8799
```

這會啟動一個真正的本機 HTTP 伺服器,終端機印出:

```
GUI-01 intake wizard serving http://127.0.0.1:8799/ for project root  
D:\\DV\\Task\\DV_Agent_Harness_L5\\v50_lab_scratch\\my_new_project
```

保留這個終端機視窗執行,另開一個終端機打真實 API:

```
curl -s http://127.0.0.1:8799/api/state
```

實際回傳(節錄):

```
{
  "schema_version": "1.0",
  "step": {"id": "new_open_project", "title": "NEW / OPEN PROJECT", "index": 0, "total": 14},
  "steps": [
    {"id": "new_open_project", "title": "NEW / OPEN PROJECT", "grounded": false},
    {"id": "verification_level", "title": "Verification Level", "grounded": false},
    {"id": "spec", "title": "Spec", "grounded": false},
    {"id": "rtl", "title": "RTL", "grounded": true},
    ...
    {"id": "user_review", "title": "User Review", "grounded": true}
  ],
  "view": {
    "grounded": false, "kind": "UNMODELED", "fields": [],
    "reason": "intake_state.py tracks no per-field record for this step; it is a navigational placeholder only, never a fabricated fact."
  },
  "overall_ready": false, "note": null
}
```

注意 `total: 14` (對應 spec 第 59 節列出的 14 步流程)以及每一步的 `grounded` 布林值 —— `grounded: false` 的步驟(例如 `new_open_project`)是誠實標示為「導覽用佔位步驟」,不是假裝有 `per-field` 資料。測完之後,回到剛才那個終端機視窗按 `Ctrl+C` 關閉伺服器。

步驟 7:查詢 intake 模式與事件字彙表

```
python -m dv_harness.cli intake-modes modes
python -m dv_harness.cli intake-events --json names
```

第一個指令列出 `FAST / STANDARD / STRICT / SIGNOFF` 四種模式各自檢查哪些類別(例如 `STANDARD` 對應現有 `evaluate_uvm_generation_ready()` 的六大 `BLOCKING_CATEGORIES`);第二個指令(記得加 `--json`,否則預設用 Python `dict` 的 `repr` 格式輸出、不是合法 JSON)列出所有 `INTAKE_*` 事件名稱,例如 `INTAKE_CREATED`、`INTAKE_BASELINED`、`INTAKE_GENERATION_READY`、`INTAKE_GENERATION_BLOCKED` 等十八個事件。

步驟 8(可選,搭配觀察):用 `--dry-run` 看看 `start` 準備做什麼

```
python -m dv_harness.cli --project-root
"D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\my_new_project" start --goal "Lab 1: intake practice" --dry-run
```

這一步不會呼叫任何真正的 LLM adapter,也不會改動狀態,只會把 `ENV_CHECK` (這個新專案目前所在的 stage)完整規劃寫成一份 JSON 報告:

```
[dv-harness] DRY_RUN stage=ENV_CHECK: plan written to ...\.dv-harness\dry_run\ENV_CHECK-20260907-125536.json (nothing executed, no state changed).
```

這讓你在不消耗真正 LLM 呼叫、不影響任何真實 `farm` 資源的情況下,預覽 `intake` 之後緊接著的第一個 stage 會做什麼。

驗證

完成本課後,請確認以下四件事都成立:

1. `find <scratch>\dv-harness` 能看到 `agents/`、`blackboard/`、`config.json`、`control.json`、`events.jsonl`、`state.json` 等完整骨架。
2. `intake-baseline list --root <scratch>` 回傳的陣列裡至少有一筆你自己 freeze 的記錄, `frozen_by` 是你設定的值(例如 `lab-student`)。
3. `intake-baseline status` (帶著同一份未變動的 `facts.json`) 回傳頂層 `"status": "VALID"`, 而不是 `INVALIDATED` 或 `UNKNOWN`。
4. `curl http://127.0.0.1:8799/api/state` 回傳的 `step.total` 是 `14`, 且陣列中確實有 `id: "rtl"`、`id: "readiness"` 等真實步驟名稱 —— 證明你打到的是真正跑起來的 wizard 後端, 不是靜態展示頁。

常見問題

1. `--facts` 一直回報 `FACTS_FILE_NOT_FOUND`, 即使我明明給了 JSON 內容? `intake-baseline` 的 `--facts` 參數吃的是檔案路徑, 不是行內 JSON 字串(這點本課實測驗證過, 容易誤會)。務必先把 facts 寫成一個 `.json` 檔, 再把檔案路徑傳進去。
2. CLAUDE.md 裡寫「沒有幫 `intake_baseline` 加 CLI verb, 只能用 `python -m`」, 但我在 `--help` 裡明明看到 `intake-baseline` 這個 verb, 怎麼回事? 這是本專案文件可能落後於程式碼的一個真實案例 —— CLAUDE.md 第 7541-7545 行記錄的是撰寫當下(`cli.py` 正被別的並行工作編輯中) 的狀態, 後來這個 verb 確實被補進 `cli.py` 了。這正是本課開頭強調的紀律: 永遠以即時的 `--help` 輸出為準, 而不是文件或記憶裡的說法, 兩者不一致時, 活的 CLI 輸出才是 ground truth。
3. `status` 指令裡的 `degraded_probe_transport.resolved` 一直是 `"none"`, `reason` 說 `LOCAL_COMMANDS_MISSING: lmutil, bqueues`, 是不是我裝壞了什麼? 不是。這代表你這台訓練機器上沒有真正的 FlexLM(`lmutil`) 或 LSF(`bqueues`) 工具鏈, harness 誠實回報「探測不到資源」而不是假裝配額充足或農場暢通 —— 這正是 Evidence Truth Rule(見 `EVIDENCE_TRUTH_RULE.md`: 「CLAUDE.md、Memory、歷史 Agent 結論與任何『合理推測』都不是 Current Evidence」) 在資源探測層面的具體體現。在沒有真實 LSF 農場連線的訓練環境下, 這是預期行為, 不是 bug。

本課使用的 `scratch` 目錄(`D:\DV\Task\DV_Agent_Harness_L5\v50_lab_scratch\`) 完全獨立於真正的 `v50` 專案, `course` 中所有指令均未修改 `v50` 本身的任何檔案或 `.dv-harness/` 狀態。所有指令、旗標名稱、輸出內容均為本課撰寫時對 `D:\DV\Task\DV_Agent_Harness_L5\v50` 現場執行 `python -m dv_harness.cli --help`、各 verb 的 `--help`, 以及實際執行取得的即時輸出, 並非憑記憶杜撰。

Lab 2: DUT RTL / Register / PHY Discovery 實作

學習目標

- 學會針對一份自建的合成(synthetic)RTL fixture,實際執行 DV Agent Harness L5 的三個真實 discovery/extraction 模組: `design_architecture_ir.py` (模組/實例樹/FSM 掃描)、`register_rtl_trace.py` (register_map 欄位對 RTL 訊號的可追溯性)、`interrupt_dma_clock_reset_extraction.py` (中斷/DMA/clock-reset 事實抽取)。
- 理解這三個工具都是**唯讀**、**declaration-level**的靜態分析,never 執行 elaboration 或 simulation,並學會分辨它們回報的「已確認」(`TRACE_CONFIRMED` / `FSM_EXTRACTION_RESOLVED`)與「誠實回報找不到證據」(`NOT_AVAILABLE`)之間的差別。
- 親身體會 CLAUDE.md 的 **No Golden-Reference Content Mining** 與 **Evidence Truth Rule**:本專案本身沒有真實 RTL tree,所以每一個範例都必須自己動手寫一份極小、明確標註為教學用的合成 fixture,絕不能挖真實 vendor RTL 或 USB_UVM_Handoff 這類參考環境的內容來充數。

前置準備

- 已完成 Lab 1(或已確認可在 `D:\DV\Task\DV_Agent_Harness_L5\v50` 下執行 `python -m dv_harness.cli --help` 並看到完整 verb 清單)。
- 這個 sandbox 已安裝 `verible-verilog-syntax` (本 Lab 已在本機驗證: `where verible-verilog-syntax` 回報 `C:\Users\peter.lin\bin\verible-verilog-syntax.exe`)。三個工具中, `design-architecture-ir` 和 `register-rtl-trace` 都依賴這個真實的 `verible` 前端去做 declaration-level parse;沒有它,這兩個工具會誠實回報 `NOT_AVAILABLE` / `BLOCKED`,而不是假裝解析成功。
- 一個可寫入的 scratch 目錄(本 Lab 全程只在 scratch 目錄下建立/讀取檔案,不會碰觸 `v50` 專案本身的任何真實檔案)。

步驟

Step 1:在 scratch 目錄建立一份極小的合成 RTL fixture

先建立一個工作目錄,再寫入一份 10~20 行、完全自創、明確標註「僅供教學」的 SystemVerilog module:

```
mkdir -p <你的scratch路徑>/lab2_fixture
```

寫入 `lab2_fixture/lab2_dma_ctrl.sv`:

```
// Synthetic teaching fixture ONLY -- not any real project's RTL.
module lab2_dma_ctrl #(
    parameter NUM_DMA_CHANNELS = 4
) (
    input logic      clk,
    input logic      rst_n,
    input logic      dma_start,
    input logic [1:0] dma_ch_sel,
    output logic      irq_dma_done,
    output logic [1:0] state
);
    logic [1:0] cur_state;
```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) cur_state <= 2'b00;
    else case (cur_state)
        2'b00: if (dma_start) cur_state <= 2'b01;
        2'b01: cur_state <= 2'b10;
        2'b10: cur_state <= 2'b00;
        default: cur_state <= 2'b00;
    endcase
end

assign state = cur_state;
assign irq_dma_done = (cur_state == 2'b10);
endmodule

```

這份 fixture 刻意包含四種可被三個工具個別驗證的元素:一個 `always @(posedge clk or negedge rst_n) + case` 構成的簡單 FSM(給 `design_architecture_ir` 的 FSM 掃描用)、`dma_start / dma_ch_sel` 兩個之後要對到 register 欄位的 port(給 `register_rtl_trace` 用)、一個含 `irq` 字樣的輸出訊號與一個 `NUM_DMA_CHANNELS` parameter(給 `interrupt_dma_clock_reset_extraction` 用)。這正是 `dv_harness_tests/test_register_rtl_trace.py` 自己遵循的慣例——用一份小型、合成、寫在測試檔裡的 fixture,而不是挖真實專案的 RTL。

Step 2:跑 `design-architecture-ir`,看它建出的模組/實例樹/FSM IR

先用 `--help` 確認即時的真實旗標(不要憑記憶猜):

```
python -m dv_harness.cli design-architecture-ir --help
```

真實輸出(節錄):

```

usage: dv-harness design-architecture-ir [-h] --rtl DAI_RTL_FILES
                                         [--top-module TOP_MODULE]
                                         [--verible-bin VERIBLE_BIN]
                                         [--out OUT] [--json]

options:
  --rtl DAI_RTL_FILES  An RTL file to include in this build (repeatable).
  --top-module TOP_MODULE
  --verible-bin VERIBLE_BIN
  --out OUT            Also write the full JSON IR to this path.
  --json

```

實際執行:

```
python -m dv_harness.cli design-architecture-ir --rtl <你的scratch路徑>/lab2_fixture/lab2_dma_ctrl.sv --json
```

這條指令會用真實的 `verible-verilog-syntax` 對你剛寫的檔案做 declaration-level parse,把 module 的 ports/parameters/signals/continuous_assigns 整理成 IR,並額外跑一個「best-effort FSM 掃描」。本 Lab 實際跑出的真實輸出(節錄關鍵欄位):

```

"fsm_extraction": {
  "status": "CANDIDATES_FOUND",
  "candidates": [

```

```
{
  "status": "FSM_EXTRACTION_RESOLVED",
  "register_name": "cur_state",
  "clock_signal": "clk",
  "reset_signal": "rst_n",
  "reset_condition_text": "!rst_n",
  "states": ["2'b00", "2'b01", "2'b10"],
  "has_default": true,
  "transitions": [
    {"from_state": "2'b00", "to_state": "2'b01", "conditional": true},
    {"from_state": "2'b01", "to_state": "2'b10", "conditional": false},
    {"from_state": "2'b10", "to_state": "2'b00", "conditional": false},
    {"from_state": "default", "to_state": "2'b00", "conditional": false}
  ]
}
]
```

同時你會看到完整的 `instance_tree` (本例只有一層, `top_modules: ["lab2_dma_ctrl"]` ,因為 `fixture` 沒有子實例)與 `duplicate_modules: []` 。這正是 CLAUDE.md 「Design Architecture IR」章節說的:這是**declaration/pattern-level extraction**,不是 **elaboration-time** 的證明——`FSM_EXTRACTION_RESOLVED` 只代表這段掃描能明確辨識出 `case-key`、狀態與轉移,並不代表已驗證這個 FSM 在實際跑動時的行為正確。

Step 3:建立一份對應的 `register_map.schema.json` ,跑 `register-rtl-trace`

先確認即時旗標:

```
python -m dv_harness.cli register-rtl-trace --help
```

真實輸出:

```
usage: dv-harness register-rtl-trace [-h] --register-map REGISTER_MAP
                                     --rtl RRT_RTL_PATHS [--strict-partial]
                                     [--json]

options:
  --register-map REGISTER_MAP  Path to a register_map.schema.json document.
  --rtl RRT_RTL_PATHS         An RTL source file to parse (repeatable).
  --strict-partial            Exit non-zero when any field's trace is TRACE_PARTIAL (ambiguous).
```

寫一份最小、符合 `dv_harness/schemas/register_map.schema.json` 的 `lab2_fixture/register_map.json` ,欄位名故意對到 `fixture` 裡的 `dma_start` / `dma_ch_sel` :

```
{
  "schema_version": "1.0",
  "source": { "kind": "programming_guide_transcription",
    "description": "Lab 2 synthetic teaching fixture -- not any real project's register map." },
  "blocks": [
    { "name": "DMA_CTRL", "base_address": "0x1000",
      "registers": [
        { "name": "DMA_CFG", "address_offset": "0x0000", "width": 32, "access": "RW",
          "reset_value": "0x0",
          "fields": [
```

```

        { "name": "dma_start", "bit_offset": 0, "bit_width": 1, "access": "RW" },
        { "name": "dma_ch_sel", "bit_offset": 1, "bit_width": 2, "access": "RW" }
    ] }
    ] }
}

```

執行:

```

python -m dv_harness.cli register-rtl-trace \
--register-map <你的scratch路徑>/lab2_fixture/register_map.json \
--rtl <你的scratch路徑>/lab2_fixture/lab2_dma_ctrl.sv \
--json

```

本 Lab 的真實輸出(節錄):

```

{
  "summary": { "total": 2, "counts": { "TRACE_CONFIRMED": 2, "TRACE_PARTIAL": 0,
                                     "TRACE_NOT_FOUND": 0, "BLOCKED": 0 } },
  "fields": [
    { "field": "DMA_CTRL.DMA_CFG.dma_start", "status": "TRACE_CONFIRMED",
      "reason": "unambiguous RTL port 'dma_start' ... matches ... and is referenced elsewhere in the
                 parsed sources. This PROVES the signal NAME exists and is wired to something; it does NOT
                 prove elaboration-time behavior..." }
  ]
}

```

兩個欄位都拿到 `TRACE_CONFIRMED`, 因為它們在 fixture 裡是「名稱唯一匹配、且被使用」的 port。注意 `reason` 欄位自己講得很清楚: 這只證明訊號名稱存在且有被接線, 不證明這個訊號實作了 register 欄位所宣稱的語意, 也不證明在真實 configuration 下這段接線是 reachable 的——這是 verible 這個純 parser 能給你的「天花板」, CLAUDE.md 對此有完整揭露。

Step 4: 跑 `interrupt-dma-clock-reset`, 看它如何誠實區分「有找到」與「沒證據」

先確認即時旗標:

```
python -m dv_harness.cli interrupt-dma-clock-reset --help
```

真實輸出:

```

usage: dv-harness interrupt-dma-clock-reset [-h] --sources SOURCES [SOURCES ...] [--json]
options:
  --sources SOURCES [SOURCES ...]  One or more real spec/programming-guide/RTL text files.
  --json

```

執行:

```

python -m dv_harness.cli interrupt-dma-clock-reset \
--sources <你的scratch路徑>/lab2_fixture/lab2_dma_ctrl.sv \
--json

```

本 Lab 的真實輸出(節錄, 六個 facet 各自獨立回報):

```

"interrupt_architecture": {
  "sources": [{"name": "irq_dma_done", "direction": "output",
    "evidence": ".../lab2_dma_ctrl.sv:9"}],
  "priority_scheme": {"status": "NOT_AVAILABLE",
    "reason": "no explicit priority statement ... a priority scheme is never inferred from the
    interrupt source list alone"},
  "masking_scheme": {"status": "NOT_AVAILABLE",
    "reason": "no explicit masking/enable statement was found ... never inferred from a register's
    name alone"}
},
"dma_architecture": {
  "channel_count": {"status": "LOADED", "value": 4, "declared_as": "NUM_DMA_CHANNELS",
    "evidence": ".../lab2_dma_ctrl.sv:3"},
  "descriptor_model": {"status": "NOT_AVAILABLE",
    "reason": "no `typedef struct packed {...} <name>;` whose closing name names a descriptor ... was
    found"}
},
"clock_reset_extension": {
  "clocks": [{"name": "clk", "evidence": ".../lab2_dma_ctrl.sv:14"}],
  "resets": [{"name": "rst_n", "active_level": "LOW", "synchronous": false, "clock": "clk",
    "description": "asynchronous reset in sensitivity list of always block at
    .../lab2_dma_ctrl.sv:14",
    "clock_resolved": "RESOLVED"}]
}

```

這裡最值得學的是: `irq_dma_done` 這個 port 因為命名符合 `irq/intr` 慣例而被正確抓出來; `NUM_DMA_CHANNELS` parameter 被正確解讀為 DMA channel 數量(`value: 4`); `clk / rst_n` 的非同步 reset 極性(`active_level: "LOW"`)從 sensitivity list 的 `negedge rst_n` 正確推得。但 `priority_scheme` 、 `masking_scheme` 、 `descriptor_model` 三個 facet 全部誠實回報 `NOT_AVAILABLE` ——因為這份 fixture 根本沒寫任何優先權/遮罩敘述句、也沒有 descriptor struct, 工具拒絕用猜的去填這些欄位。這正是 CLAUDE.md 明文要求的行為: 「never infer a priority scheme or channel count that is not written down」。

驗證

- ☐ `design-architecture-ir --json` 的輸出裡 `fsm_extraction.candidates[0].status` 是 `FSM_EXTRACTION_RESOLVED`, 且 `states` 恰好列出 `2'b00 / 2'b01 / 2'b10` 三個狀態。
- ☐ `register-rtl-trace --json` 的 `summary.counts.TRACE_CONFIRMED` 等於 2, `TRACE_NOT_FOUND` 與 `BLOCKED` 都是 0。
- ☐ `interrupt-dma-clock-reset --json` 的 `dma_architecture.channel_count.value` 等於 4, 且 `priority_scheme.status` 與 `masking_scheme.status` 都是 `NOT_AVAILABLE` (而不是被工具「猜」出一個假的優先權配置)。
- ☐ 三個指令的 exit code 皆為 0(可用 `echo $?` 確認), 代表 IR 成功建出, 不是走 `NOT_AVAILABLE / BLOCKED` 的失敗路徑。

常見問題

1. 為什麼 `priority_scheme / masking_scheme / descriptor_model` 一直是 `NOT_AVAILABLE`, 是不是我的指令下錯了? 不是。這是設計上的行為: `interrupt_dma_clock_reset_extraction.py` 只認得三種明確的英文句型(如 "X has the highest priority")或 `typedef struct packed {...} <name>_desc;` 這種結構, 絕不會從中斷來源清單或暫存器命名去「腦補」出一個優先權方案或 descriptor 格式。想看到 `LOADED` 狀態, 必須在 fixture 文字裡真的寫一句符合句型的敘述。

2. 在沒有 `verible-verilog-syntax` 的機器上跑會怎樣? `design-architecture-ir` 會回報整份 IR 的 `status:` `"NOT_AVAILABLE"` (或個別檔案 `status: "PARSE_FAILED"`, 視情況而定), `register-rtl-trace` 每個欄位都會落到 `BLOCKED` 而不是 `TRACE_NOT_FOUND` ——CLAUDE.md 特別強調這兩者要分開: `BLOCKED` 代表「根本沒能真的去查」, `TRACE_NOT_FOUND` 代表「真的查了、確實沒找到」, 絕不可混為一談。本機因為已裝好 `verible` (`C:\Users\peter.lin\bin\verible-verilog-syntax.exe`), 所以三個指令都走到了真實的解析路徑。
 3. 這三個工具能不能直接拿去分析真實專案的 RTL? 可以, 用法完全一樣, 只要把 `--rtl / --register-map / --sources` 換成真實檔案路徑即可。但務必記得 CLAUDE.md 的兩條硬性規則: 這幾個工具給出的都是 `declaration-level` 的靜態證據 (parser 讀取到什麼就報什麼), 從來不是 `elaboration` 或模擬層級的行為證明; 而且依照 `No Golden-Reference Content Mining` 原則, 絕不能拿任何完成度高的參考環境 (如 `USB_UVM_Handoff`) 的內容去產生「看起來像被工具找到」的假 `fixture` ——每個教學/測試用的合成範例都必須像本 Lab 一樣, 明確標註為「`synthetic fixture only`」。
-

Lab 3: VIP Discovery 與 API 可驗證性檢查

學習目標

- 學會用 `dv_harness.vip_symbol_index` 對一份 VIP 原始碼樹建立「符號索引」(symbol index)——只記錄 class/method/config field 的宣告與 `file:line` 位置,絕不保留任何方法本體(method body),藉此在不違反 CLAUDE.md Tier 1「NEVER-VIP-SOURCE」禁令的前提下,讓 VIP 資訊可以進入 agent 的分析流程。
- 學會用 `dv_harness.vip_capability_extraction` 把符號索引進一步分類成五種 VIP 能力 IR(Config / Transaction / Scenario Pattern / Checker / Coverage),並理解每一項分類都帶有可信度標記 (qualification),而非武斷的「猜測」。
- 學會用 `dv_harness.vip_api_card` (也可透過 `dv-harness vip-api-check`) 驗證「已生成的測試序列 (sequence)」裡引用的 VIP API 是否真的存在於索引中——這正是本專案 Spec 187 節「API 若無法被證明存在,必須回報 UNKNOWN/BLOCKED,絕不可讓幻覺 API 靜默地被產生」的具體落地機制。
- 親手觸發一次「幻覺 VIP API」被攔截(BLOCKED)的實際案例,理解這條防線如何避免 agent 憑空捏造 VIP class/method。

前置準備

- 已確認本專案已內建一份合成(synthetic)VIP 原始碼範例,位於 `examples/asset_processing/inputs/vip_src/svt_demo_pkg.sv`。這份檔案的檔頭註解明確聲明:它「不是任何真實的 Synopsys VIP,也不含任何廠商程式碼」,是刻意寫成長得像 `svt_` 前綴的 UVM VIP package(含 config/sequence/driver/monitor/callback 等 class),專門用來讓 `vip_symbol_index.py` 可以在真實文字上被完整演練,而不需要一份有授權限制的真實 VIP 原始碼進入本 repo。
- 本 Lab 全程只做唯讀分析(讀取原始碼、產生 JSON 分析結果),不會修改 `v50` 專案本身的任何檔案——所有輸出都寫到你自已建立的 scratch 目錄。
- 建議先建立一個乾淨的工作目錄:

```
mkdir -p /tmp/lab3_scratch
SCRATCH=/tmp/lab3_scratch
```

(Windows PowerShell 使用者可改用 `New-Item -ItemType Directory -Force -Path C:\temp\lab3_scratch` 並設定 `$SCRATCH="C:\temp\lab3_scratch"`。)

步驟

步驟 1: 確認三個模組的即時說明(CLI ground truth)

在寫任何指令之前,先用 `--help` 確認真正的參數名稱——這是本專案避免文件過時、避免對 CLI 語法「憑記憶猜測」的鐵律。

```
python -m dv_harness.vip_capability_extraction --help
python -m dv_harness.vip_api_card --help
```

你會看到什麼:兩者都輸出完整的 `argparse` 說明。特別注意 `vip_capability_extraction` 需要 `--index` (一份 `vip_symbol_index` JSON 文件),而 `vip_api_card` 同時需要 `--source` (要驗證的已生成 `.sv` / `.svh`)與 `--index`。

重要: `dv_harness.vip_symbol_index` 這個模組沒有自己的 CLI 入口——執行 `python -m dv_harness.vip_symbol_index --help` 只會安靜地以 exit code 0 結束,不會印出任何 usage 說明(可自行驗證)。它是一個純函式庫模組,設計上是被其他 Python 程式(或未來的生成器)以 `import` 方式呼叫,而不是給人在終端機直接執行。這點在後續步驟會直接體現。

步驟 2:用 Python API 建立符號索引(vip_symbol_index)

由於 `vip_symbol_index` 沒有 CLI,我們用它公開的三個函式: `build_symbol_index()`、`save_symbol_index()`、`find_symbol()`。

```
python -c "  
from dv_harness.vip_symbol_index import build_symbol_index, save_symbol_index, find_symbol  
  
doc = build_symbol_index(  
    ['examples/asset_processing/inputs/vip_src'],  
    protocol='demo',  
    relative_to='examples/asset_processing/inputs',  
)  
save_symbol_index(doc, '\$SCRATCH/vip_symbol_index.json')  
print('classes_indexed:', doc['stats']['classes_indexed'])  
print('methods_indexed:', doc['stats']['methods_indexed'])  
print(find_symbol(doc, 'apply_preset'))  
"
```

`build_symbol_index()` 的第一個參數是要掃描的 VIP 原始碼根目錄清單, `protocol` 是呼叫端自己指定的協定名稱(本專案刻意規定:協定名稱絕不可以從資料夾名稱「猜」出來,必須由使用者明確傳入,避免產生錯誤的隱含事實)。`relative_to` 讓輸出裡的檔案路徑變成相對路徑,這樣同一份 VIP 原始碼在不同機器上索引出來的 JSON 才會是逐位元組相同、可 diff 的。

實際輸出(本次真實執行結果):

```
classes_indexed: 7  
methods_indexed: 13  
[{'kind': 'function', 'name': 'svt_demo_cfg.apply_preset', 'arguments': '(input int preset_id)',  
  'file': 'vip_src/svt_demo_pkg.sv', 'line': 26}]
```

`find_symbol()` 回傳的是「宣告位置」而非方法本體——你可以打開 `vip_symbol_index.json` 檢查,裡面每一個 method 只有 `name` / `kind` / `return_type` / `arguments` / `file` / `line`,完全不含程式邏輯(例如 `apply_preset` 函式體裡真正的 `if (preset_id > 0) begin ... end` 邏輯完全沒有被存下來)。

注意:本專案 repo 中另外還有一份已經預先產生好的範例輸出檔

`examples/asset_processing/generated/vip_symbol_index.json`,裡面記錄的是 `classes_indexed: 6`。這是因為該檔案是在原始碼加入第 6 個 class(`svt_demo_report_cb`,一個 UVM callback class)之前產生的舊快照,尚未重新生成。這正是一個很好的教材:已產生的產物 (generated artifact) 只是某個時間點的快照,永遠要以「重新對照即時原始碼跑一次」為準,這也是本 Lab 系列反覆強調的 grounding 紀律。

步驟 3:用 vip_capability_extraction 把索引分類成五種能力 IR

```
python -m dv_harness.vip_capability_extraction --index "$SCRATCH/vip_symbol_index.json" --json
```

實際輸出(節錄,真實執行結果):

```
{
  "status": "CLASSIFIED",
  "protocol": "demo",
  "items": [
    {
      "ir_type": "VIPConfigIR",
      "class_name": "svt_demo_cfg",
      "qualification": "INFERRED_FROM_NAMING",
      "basis": "NAME_ONLY",
      ...
    },
    {
      "ir_type": "VIPTransactionIR",
      "class_name": "svt_demo_transaction",
      "qualification": "INFERRED_FROM_NAMING",
      "basis": "NAME_AND_INHERITANCE_AGREE",
      ...
    },
    ...
  ]
}
```

每一筆分類都同時看兩個獨立線索:**命名規則**(class 名稱最後一段是否符合 `_cfg` / `_transaction` / `_seq` 等慣例)與**繼承鏈**(是否真的繼承自 `uvm_sequence_item`、`uvm_monitor` 等已知 UVM base class)。 `qualification` 欄位是關鍵: `INFERRED_FROM_NAMING` 表示只有較弱的證據支持(預設值),只有當呼叫端額外提供「這個 class 真的被專案原始碼/VIP Examples/使用手冊引用過」的引證時,才會被升級為 `PROJECT_PROVEN`、`VIP_EXAMPLE_MATCHED` 或 `VIP_DOCUMENTED`。若命名與繼承鏈**互相矛盾**(例如名稱像 `config` 但繼承鏈卻像 `scenario pattern`),該筆會被獨立列在 `ambiguous` 清單、狀態鎖定為 `UNKNOWN`,絕不會被硬塞進五種分類的任何一種——這就是本專案「寧可誠實回報不確定,也不做無根據的猜測」的一貫紀律。

步驟 4:用 `vip_api_card` 驗證一份「已生成的測試序列」引用的 VIP API 是否真實存在

本 repo 內建一份合成的測試 fixture,模擬本專案生成器真正會產出的「virtual sequence 呼叫 VIP API」樣式:
`dv_harness_tests/fixtures/vip_api/demo_env_seq.sv`。

```
python -m dv_harness.vip_api_card \
  --source dv_harness_tests/fixtures/vip_api/demo_env_seq.sv \
  --index "$SCRATCH/vip_symbol_index.json"
echo "exit code: $?"
```

實際輸出(真實執行結果):

```
VIP API validation: PROVEN
protocol: demo
files scanned: 1
VIP naming scope: svt_
cards: PROVEN=8, BLOCKED=0, UNPROVABLE=0, OUT_OF_SCOPE=2

PROVEN (VIPApiCard -> real VIP source location):
  svt_demo_cfg -> vip_src/svt_demo_pkg.sv:18
  svt_demo_agent -> vip_src/svt_demo_pkg.sv:94
  svt_demo_transaction -> vip_src/svt_demo_pkg.sv:41
  svt_demo_cfg.set_defaults -> vip_src/svt_demo_pkg.sv:24
```

```
svt_demo_cfg.apply_preset -> vip_src/svt_demo_pkg.sv:26
svt_demo_cfg.wait_for_ready -> vip_src/svt_demo_pkg.sv:33
svt_demo_transaction::type_id -> vip_src/svt_demo_pkg.sv:41
svt_demo_transaction.convert2string -> vip_src/svt_demo_pkg.sv:46

exit code: 0
```

每一筆 **PROVEN** 都直接指向真實原始碼的 `file:line` ——這就是「可驗證性」的具體意義:不是相信 agent 說的話,而是回頭指到一個人可以親自打開來核對的真實宣告位置。

```
也可以改用 dv-harness CLI 前門執行完全等效的檢查: dv-harness vip-api-check --source
dv_harness_tests/fixtures/vip_api/demo_env_seq.sv --index "$SCRATCH/vip_symbol_index.json"
```

步驟 5:親手製造一次「幻覺 API」,看 **BLOCKED** 機制實際攔截它

複製一份 fixture,故意把一個真實存在的方法名稱打錯(模擬 LLM 幻覺出一個不存在的 VIP API):

```
cp dv_harness_tests/fixtures/vip_api/demo_env_seq.sv "$SCRATCH/bad_env_seq.sv"
sed -i 's/apply_preset/apply_prezet/' "$SCRATCH/bad_env_seq.sv"

python -m dv_harness.vip_api_card \
  --source "$SCRATCH/bad_env_seq.sv" \
  --index "$SCRATCH/vip_symbol_index.json"
echo "exit code: ${?}"
```

實際輸出(真實執行結果):

```
VIP API validation: BLOCKED
protocol: demo
files scanned: 1
VIP naming scope: svt_
cards: PROVEN=7, BLOCKED=1, UNPROVABLE=0, OUT_OF_SCOPE=2

BLOCKED -- cannot be proven, must not be generated:
svt_demo_cfg.apply_prezet [METHOD_NOT_DECLARED_IN_INDEXED_VIP_CHAIN]
  cited at ../bad_env_seq.sv:30: cfg.apply_prezet(2);

PROVEN (VIPApiCard -> real VIP source location):
svt_demo_cfg -> vip_src/svt_demo_pkg.sv:18
...

exit code: 1
```

`svt_demo_cfg.apply_prezet` 被判定為 `METHOD_NOT_DECLARED_IN_INDEXED_VIP_CHAIN`:程式已經沿著 `svt_demo_cfg` 完整的繼承鏈找過,索引裡就是沒有這個方法,而且整條鏈的所有 base class 都在索引「封閉世界」(closed world)之內(即繼承鏈上沒有走到索引以外、未知的 base class),所以這不是「不確定」,而是可以確信地判定為**不存在**——因此狀態是 **BLOCKED** 而非較保守的 **UNPROVABLE**,回傳碼也是非 0(**1**),代表這是一個會讓 pipeline 失敗的硬性錯誤。

驗證

完成本 Lab 後,請確認以下四點:

1. `$SCRATCH/vip_symbol_index.json` 存在,且 `stats.classes_indexed` 為 7 (若你看到 6,代表你不小心用到了舊的 `examples/asset_processing/generated/vip_symbol_index.json`,而不是自己重新產生的版本)。
2. 打開該 JSON,任意挑一個 method(例如 `apply_preset`),確認裡面只有 `name / kind / arguments / file / line` 這類宣告層級欄位,完全沒有任何函式本體的邏輯文字(例如不會出現 `if (preset_id > 0)`)。
3. `vip_api_card` 對乾淨的 `demo_env_seq.sv` 執行結果為 `PROVEN`,回傳碼 0;對你自己改壞的 `bad_env_seq.sv` 執行結果為 `BLOCKED`,回傳碼 1,且 `BLOCKED` 清單裡精準點出你改壞的那一行呼叫(`bad_env_seq.sv:30`)。
4. `vip_capability_extraction` 的輸出中,至少能認出一筆 `VIPConfigIR` (對應 `svt_demo_cfg`)與一筆 `VIPTransactionIR` (對應 `svt_demo_transaction`),且每一筆的 `qualification` 都是 `INFERRED_FROM_NAMING` (因為本次沒有額外提供 `--project-source / --example-source / --user-guide-reference-md` 引證)。

常見問題

- Q:我對一份真正的、受授權保護的 VIP 原始碼樹(例如公司內部的 Synopsys VIP)跑 `vip_symbol_index`,會不會把原始碼內容洩漏進 agent 的對話 context? A:不會,這正是這個模組存在的核心設計目的。CLAUDE.md 的 Context Budget 規則把「VIP 原始碼全文」列為 Tier 1(`NEVER-VIP-SOURCE`,一律被 `context-budget-guard` hook 擋下、禁止被讀進 context),而 `vip_symbol_index.py` 產生的索引只保留宣告與位置、明確拒絕保留任何方法本體——`assert_no_bodies_retained()` 這個函式就是把這個不變量變成一個「可被程式檢查」的斷言,而不只是寫在文件裡的承諾。VIP Examples 目錄下的參考測試環境與 `.f` `filelist` 則是明確被列為可讀的例外。
 - Q: `vip_capability_extraction` 分類出來的東西可以直接拿去當作「已驗證的 VIP API 事實」使用嗎? A:不行,除非它的 `qualification` 是 `PROJECT_PROVEN`、`VIP_EXAMPLE_MATCHED` 或 `VIP_DOCUMENTED` (這三者都要求真實的 `file:line` 或文件章節引證)。預設值 `INFERRED_FROM_NAMING` 只代表「這個 class 的名稱與繼承鏈長得像某種能力」,是一個待驗證的猜測,不是事實。而且這個分類器本身有一個已知的結構性限制:它只判斷 method 呼叫與 class 型別引用,對於單純的屬性存取(例如 `cfg.some_field`)完全不做判斷——因為索引器只針對「受限的資料型別集合」建立欄位索引,欄位不在索引裡不代表欄位真的不存在。
 - Q:為什麼 `apply_preset` (我打錯的方法名)被判為 `BLOCKED`,而不是比較保守的 `UNPROVABLE`?兩者差在哪?
A: `vip_api_card.py` 的設計刻意把 `BLOCKED` 的條件收得很窄:只有當「接收端型別本身確定在索引內」、「整條繼承鏈完全走到底、沒有走出索引之外的未知 base class(也就是所謂的封閉世界)」、「該成員確定不在整條鏈的任一層」、且「這也不是任何已知的 UVM/SystemVerilog base library 方法」時,才會判 `BLOCKED`。只要繼承鏈中有一個 base class 走出了索引範圍(開放世界/open inheritance chain),無法排除該方法其實宣告在那個未知 base class 裡,就只能誠實回報 `UNPROVABLE` (對應 spec 187 節所說的「無法證明時回報 UNKNOWN」),而不是武斷地判定它不存在。這個區分正是為了避免「因為我們看不到完整資訊,就錯誤地把一個其實存在的 API 判為幻覺」這種假陽性。
-

Lab 4: 產生一個 Subsystem-Mode UVM 驗證環境

學習目標

- 理解 CLAUDE.md 規定的「Environment Generation Mode」二選一機制:任何 CREATE ENVIRONMENT 動作開始前,都必須先決定是 **SUBSYSTEM_MODE**(建立單一 protocol/subsystem 環境)還是 **SYSTEM_LEVEL_MODE**(組合已註冊完成的多個 subsystem 成 SoC 環境),並學會用真實輸出辨認系統做出了哪個決定。
- 學會使用本專案唯一真正的 CREATE ENVIRONMENT 進入點——
`tools/generate_protocol_uvm_environment.py` (實際邏輯分派到 `dv_harness/uvm_generator/create_environment.py`)——並看懂它回傳的 JSON 結構(`environment_mode`、`environment_mode_decision`、`protocol_model`、`structural_lint` 等欄位)。
- 認識一個真實 manifest(生成請求)大致長什麼樣子,並知道去哪裡找本專案自己產生過的真實範例輸出(`examples/` 目錄)而不是憑空想像。
- 理解為什麼 **SYSTEM_LEVEL_MODE** 在沒有「已透過 **SUBSYSTEM_MODE** 走完 SIGNOFF PASS 並登記進 registry」的前提下,系統會直接拒絕生成,而不是安靜地生成一個殘缺結果。

前置準備

- 已經完成過前面幾個 Lab,熟悉在 `D:\DV\Task\DV_Agent_Harness_L5\v50` 目錄下用 `python -m ...` 執行 `dv_harness` 的模組化 CLI。
- 這個 Lab 全程屬於 **LOCAL_ANALYSIS**(純本地讀寫檔案,不連伺服器、不跑 VCS/模擬),符合 CLAUDE.md 的 Execution Mode Gate,不需要事先宣告 **REMOTE_EXECUTION**。
- 了解專案的「No Golden-Reference Content Mining」規則: `reference/USB_UVM_Handoff` 這類真實 `reference` 環境只能拿來做「事後結構符合度比對」,絕對不可以是產生器讀取內容的來源。這條規則會直接影響你在真實專案中怎麼寫 manifest(VIP 行為內容必須來自 VIP 手冊/範例/原始碼、DUT 行為內容必須來自 RTL/PHY 文件,不能照抄別人已完成的環境)。
- (選讀) `verible-verilog-syntax` 是否安裝在你的機器上不影響本 Lab 能否跑完——生成器內建的 `structural lint` 在找不到 `verible` 時會誠實回報 `NOT_AVAILABLE`,而不是假裝通過。

步驟

Step 1:查看真實的 CLI 語法(永遠先做這一步,不要憑記憶猜 flag)

```
python -m tools.generate_protocol_uvm_environment --help
```

真實輸出(已在本次撰寫前實際執行確認):

```
usage: python.exe -m tools.generate_protocol_uvm_environment
       [-h] --manifest MANIFEST --out OUT [--root ROOT]

options:
  -h, --help            show this help message and exit
  --manifest MANIFEST
  --out OUT
  --root ROOT
```

只有三個參數: `--manifest` (生成請求 JSON)、`--out` (輸出目錄)、`--root` (專案根目錄,預設是目前工作目錄,subsystem registry 就是從這裡的 `.dv-harness/soc-composer/subsystem_environment_registry.json` 讀出來的)。這個腳本本身不是真正的產生器,它只是一個薄殼,實際邏輯全部在 `dv_harness/uvvm_generator/create_environment.py` 的 `create_environment()`。

Step 2:檢視本專案自己真實產生過的範例輸出

不要憑空猜 manifest 該怎麼寫,先看專案自己留下的真實產物。`examples/` 目錄下每一個 `generated_usb_real_evidence_v*` 都是這個 CLI 真的跑過、留下來的輸出:

```
ls examples/
```

會看到 `generated_usb_real_evidence_v1` 到 `v12` (每一版是同一個 USB manifest 逐步加證據的演進紀錄)、`generated_pcie_uvm_env` (PCIe skeleton)。注意: `examples/NOTICE_SCAFFOLDING_ONLY.md` 明確聲明這些是「scaffolding only」——沒有一份反映真的模擬跑過或真的 signoff,不能被引用為驗證證據。

看最簡單的 `v1` 版本實際長出來的檔案:

```
find examples/generated_usb_real_evidence_v1 -type f | sort
```

真實輸出:

```
examples/generated_usb_real_evidence_v1/bind/dv_uvm_hook.svh
examples/generated_usb_real_evidence_v1/bind/usb_uvm_bind_inst.sv
examples/generated_usb_real_evidence_v1/environment_manifest.json
examples/generated_usb_real_evidence_v1/filelist/dv_uvm_files.f
examples/generated_usb_real_evidence_v1/manifest_inputs/usb_bind_topology.json
examples/generated_usb_real_evidence_v1/manifest_inputs/usb_env_manifest.json
examples/generated_usb_real_evidence_v1/manifest_inputs/usb_patterns.json
examples/generated_usb_real_evidence_v1/patterns_registry/...
examples/generated_usb_real_evidence_v1/tb/env/usb_config.sv
examples/generated_usb_real_evidence_v1/tb/env/usb_coverage.sv
examples/generated_usb_real_evidence_v1/tb/env/usb_env.sv
examples/generated_usb_real_evidence_v1/tb/env/usb_env_pkg.sv
examples/generated_usb_real_evidence_v1/tb/env/usb_scoreboard.sv
examples/generated_usb_real_evidence_v1/tb/seq/usb_base_vseq.sv
examples/generated_usb_real_evidence_v1/tb/seq/usb_virtual_sequencer.sv
examples/generated_usb_real_evidence_v1/tb/tests/usb_base_test.sv
examples/generated_usb_real_evidence_v1/tb/tests/usb_connect_test.sv
examples/generated_usb_real_evidence_v1/tb/top/tb_top.sv
```

`manifest_inputs/usb_env_manifest.json` 是「輸入」(你要餵給 `--manifest` 的東西),其餘是「輸出」。真實輸入 manifest 的頂層 key 大致是: `protocol`、`role`、`port_count`、`interfaces` (DUT 訊號清單)、`vip.package_imports`、`clocks`、`resets`、`smoke_tests`、`scoreboard_rules`、`coverage_points`、`vip_components` (每個 UVM agent/env 的類別與依賴關係,附真實 file:line 證據)、`connections` (connect_phase 的接線)。這份 `v12` manifest 本身超過 800 行、每個欄位都附著真實的 RTL/DOC/VIP 檔案行號引用——這正是 CLAUDE.md「No Golden-Reference Content Mining」規則要求的證據密度,不是隨便編的。

Step 3:確認輸出端的 `environment_manifest.json` 長什麼樣

```
python -c "
import json
```

```
d = json.load(open('examples/generated_usb_real_evidence_v1/environment_manifest.json',
encoding='utf-8'))
print(list(d.keys()))
print(d['generated_files'])
"
```

真實輸出:

```
['_evidence_notes', 'protocol', 'role', 'interfaces', 'vip', 'clocks', 'resets', 'smoke_tests',
'scoreboard_rules', 'coverage_points', 'generated_files', 'qualification_status']
['filelist/dv_uvm_files.f', 'tb/env/usb_config.sv', 'tb/env/usb_coverage.sv', 'tb/env/usb_env.sv',
'tb/env/usb_env_pkg.sv', 'tb/env/usb_scoreboard.sv', 'tb/seq/usb_base_vseq.sv',
'tb/seq/usb_virtual_sequencer.sv', 'tb/tests/usb_base_test.sv', 'tb/tests/usb_connect_test.sv',
'tb/top/tb_top.sv']
```

這個 `environment_manifest.json` 其實就是輸入 manifest 加上 `generated_files` 清單,寫回輸出目錄,作為這次生成的自我描述紀錄。

Step 4:自己動手,在 scratch 目錄跑一次真的 SUBSYSTEM_MODE 生成

這一步會真的執行生成器並寫出檔案,絕對不要把 `--out` 指到 v50 專案目錄裡——一律用系統暫存目錄。先寫一個最小 manifest:

```
cat > /tmp/lab4_min_manifest.json << 'EOF'
{
  "protocol": "usb",
  "role": "device",
  "interfaces": [{"name": "usb0_dp"}],
  "vip": {"package_imports": ["svt_usb_uvm_pkg"]},
  "clocks": [{"name": "clk"}],
  "resets": [{"name": "rst_n", "polarity": "active_low"}],
  "smoke_tests": [{"name": "connect"}]
}
EOF
python -m tools.generate_protocol_uvm_environment \
  --manifest /tmp/lab4_min_manifest.json \
  --out /tmp/lab4_scratch/env_out
```

這是本次撰寫本 Lab 時實際跑出來的真實輸出(節錄關鍵欄位):

```
{"status": "OK",
"environment_mode": "SUBSYSTEM_MODE",
"generated_files": ["filelist/dv_uvm_files.f", "tb/env/usb_assertions.sv",
"tb/env/usb_config.sv", "tb/env/usb_coverage.sv", "tb/env/usb_env.sv",
"tb/env/usb_env_pkg.sv", "tb/env/usb_scoreboard.sv",
"tb/seq/usb_base_vseq.sv", "tb/seq/usb_virtual_sequencer.sv",
"tb/tests/usb_base_test.sv", "tb/tests/usb_connect_test.sv",
"tb/top/tb_top.sv", "environment_manifest.json"],
"environment_mode_decision": {
  "resolved": true, "environment_mode": "SUBSYSTEM_MODE",
  "requested_subsystems": ["usb"], "needs_subsystem_mode_first": false,
  "evidence": "environment_mode_router.resolve_environment_mode: exactly 1
  requested subsystem ('usb') -> SUBSYSTEM_MODE ..."},
"protocol_model": {
```

```

"protocol": "USB_2_3x", "capability_status": "DUT_PROVEN",
"status": "NO_PROTOCOL_MODEL_FOR_PROTOCOL",
"reason": "protocol_capability.PROTOCOL_CAPABILITIES declares no
protocol-model module for USB_2_3x; only the protocol-agnostic
skeleton applies"},
"structural_lint": {
  "status": "PASS", "verible_version": "v0.0-4150-gfe58e708",
  "classes_analyzed": 8, "error_count": 0, "warning_count": 1,
  "findings": [{"rule": "CONFIG_DB_GET_WITHOUT_SET", "severity": "WARNING",
    "subject": "cfg", "message": "uvm_config_db field 'cfg' is read here
    but never set anywhere in the analysed environment (guarded get with
    a fallback)}]}}

```

三個重點必須看懂:(1) `environment_mode` 是系統自己算出來的,因為只給了一個 `protocol` 欄位、沒給 `requested_subsystems`, 路由器判定為 `SUBSYSTEM_MODE`; (2) `protocol_model.status` 是 `NO_PROTOCOL_MODEL_FOR_PROTOCOL` ——USB 目前在這個框架裡沒有專屬的協定模型層(如 PCIe 的 LTSSM), 只會拿到協定無關的骨架, 這是誠實揭露而不是隱藏; (3) `structural_lint` 是生成完後自動對剛產生的 SystemVerilog 跑的 verible 靜態檢查, PASS 但有 1 個 WARNING——這正是 CLAUDE.md 「UVM Structural Lint」段落描述的機制, 在真正進 VCS compile 之前先抓結構性問題。

Step 5: 觀察 SYSTEM_LEVEL_MODE 在沒有已註冊 subsystem 時的真實拒絕行為

```

cat > /tmp/lab4_sys_manifest.json << 'EOF'
{
  "requested_subsystems": ["usb", "pcie"],
  "soc_name": "demo_soc"
}
EOF
python -m tools.generate_protocol_uvm_environment \
  --manifest /tmp/lab4_sys_manifest.json \
  --out /tmp/lab4_scratch/soc_out
echo "EXIT_CODE=$?"

```

真實輸出:

```

{"status": "SUBSYSTEM_MODE_REQUIRED_FIRST",
"detail": {"missing_subsystems": ["usb", "pcie"],
  "next_action": "invoke SUBSYSTEM_MODE builder for missing required
  subsystem(s) ['usb', 'pcie'], register it/them, then return to
  SYSTEM_LEVEL_MODE (environment_mode_policy.json
  missing_subsystem_behavior)",
  "decision": {"environment_mode": "SYSTEM_LEVEL_MODE",
    "needs_subsystem_mode_first": true, ...}}}
EXIT_CODE=2

```

這個 exit code 2 是故意的: `requested_subsystems` 有兩個以上, 路由器判定應該走 `SYSTEM_LEVEL_MODE`, 但真正的 `subsystem_environment_registry.json` (只有在某個 subsystem 走完 `SUBSYSTEM_MODE` 並且真的通過 SIGNOFF PASS 之後, 才由 `engine.py` 寫入) 裡沒有 `usb`、`pcie` 這兩筆登記, 於是系統直接拋出 `SubsystemModeRequiredError`, 而不是靜靜地少生成一個 subsystem——這正是 CLAUDE.md 「若在 `SYSTEM_LEVEL_MODE` 缺少所需 subsystem, 先用 `SUBSYSTEM_MODE` 建好再回來組合」這條規則被實作成程式碼強制執行的樣子。本沙盒訓練環境沒有真正跑過完整的 SIGNOFF 流程, 所以這裡永遠只能示範到「被正確拒

絕」這一步;要真正做出 SYSTEM_LEVEL_MODE 的組合結果,需要先在一個真實專案裡把每個 subsystem 走完整條 Graph(INTAKE → ... → SIGNOFF PASS),這超出本 Lab 範圍。

驗證

完成本 Lab 後,請確認:

- ☐ `python -m tools.generate_protocol_uvm_environment --help` 能正常印出三個參數,且你能解釋 `--root` 的作用(讀取哪個目錄下的 subsystem registry)。
- ☐ 你能在 `examples/` 目錄下指出至少兩個真實生成過的範例,並說出 `manifest_inputs/` 與其餘輸出檔案的差別。
- ☐ Step 4 的指令在你自己的機器上跑出 `"status": "OK"`,且 `/tmp/lab4_scratch/env_out/tb/env/` 底下真的多出 `usb_env.sv` 等檔案。
- ☐ 你能解釋為什麼 `protocol_model.status` 顯示 `NO_PROTOCOL_MODEL_FOR_PROTOCOL`,而不是誤以為 USB 沒有任何模型層支援。
- ☐ Step 5 的指令回傳 exit code 2 且訊息是 `SUBSYSTEM_MODE_REQUIRED_FIRST`,你能說出「為什麼系統選擇拒絕而不是生成一半」。

常見問題

1. 「為什麼 structural_lint 顯示 verible 版本,但我的機器上沒裝 verible 也想跑這個 Lab?」沒關係——`uvm_structural_lint.py` 在偵測不到 `verible-verilog-syntax` 執行檔時,會回傳 `"status": "NOT_AVAILABLE"` 並附上真實原因,而不是假裝跑過並回傳 PASS(CLAUDE.md 明確要求「NOT_AVAILABLE 絕不可以被誤讀成 PASS」)。這不會擋住環境生成,只是少了這一層靜態檢查的證據。
 2. 「manifest 裡的 `vip_components / connections / scoreboard_rules` 欄位到底可以怎麼寫?我可以照抄 USB_UVM_Handoff 嗎?」絕對不可以直接照抄內容拿去產生別的專案的環境——CLAUDE.md 的「No Golden-Reference Content Mining」規則明確禁止「從一個已完成的參考環境挖內容 → 讓產生器重現它 → 驗證它符合」這種做法,即使目標是「100% 對齊某個真實環境」也不行。真正的做法是:VIP 相關內容(agent/sequence/API)必須從 VIP 範例、VIP user guide、VIP 原始碼取得證據;DUT 相關內容(register、時序)必須從 DUT RTL、PHY 文件、programming guide 取得證據。已完成的參考環境只能拿來做「事後結構符合度比對」,不能是內容來源。
 3. 「Step 5 為什麼不能直接示範出一個真的 SYSTEM_LEVEL_MODE 組合結果?」因為 `compose_soc_environment()` 只會從 `.dv-harness/soc-composer/subsystem_environment_registry.json` 這份真實登記檔讀取候選 subsystem,而這份檔案只有 `engine.py` 的 `_persist_subsystem_registry_entry()` 在某個 subsystem 真的走完整條驗證 Graph、通過 gate-validated SIGNOFF PASS 之後才會寫入——這是刻意設計成「無法被呼叫端偽造」的防護(呼叫端不能把一個尚未驗證過的 subsystem 硬塞進系統級組合)。本沙盒環境沒有可用的真實多 subsystem 專案狀態,所以只能誠實示範到「正確地被拒絕」,而不是編造一個假裝成功的組合輸出。
-

Lab 5: AMBA 匯流排拓樸與交易分析上機練習

學習目標

- 理解 `amba_fabric_discovery.py` (AMBA 拓樸探索/AMBA-7..14)、`amba_port_registry.py` (AMBA_PORT_REGISTRY/AMBA-22)、`amba_transaction_ir.py` (Transaction IR/SYOSCB-9..10)三個模組在 AMBA 匯流排分析流程中各自的職責,以及它們如何串成一條「拓樸探索 → Port Registry → Transaction IR」的真實資料鏈。
- 親手用一個最小的合成 AMBA4 SoC RTL fixture,實際呼叫這三個模組的真實 Python API,並確認這條 pipeline 在 real `verible-verilog-syntax` 解析下能跑出真實、可驗證的輸出。
- 弄清楚一件容易誤解的事:這三個檔案本身**沒有**獨立的 `python -m` CLI 前門,並學會這個專案裡「函式庫模組」與「有 CLI 前門的下游模組」的分界線在哪裡、要怎麼驗證。
- 用真實的 `dv-harness amba-readiness-gates` CLI verb,把手上做出來的 IR 對應到 `AMBA_PORT_REGISTRY_READY` 等九個 composite readiness gate 的條件名稱。

前置準備

- 已經完成過前面幾個 Lab,熟悉 `python -m dv_harness.cli --help` 的基本操作與 `--help` 的查閱習慣。
- 本機已安裝 `verible-verilog-syntax` 並在 PATH 上——這個環境已經驗證過: `verible-verilog-syntax --version` 回報 `v0.0-4150-gfe58e708`。如果你的機器上沒有這支 binary,這三個模組的官方測試套件會用 `pytest.mark.skipif` 誠實跳過,絕對不會假造一份「看起來像通過」的結果。
- 你在 `v50` 專案根目錄下操作,並知道如何用 `PYTHONPATH` 執行專案外部的 scratch script(因為等一下你會發現,這三個模組本身根本沒有掛 CLI 入口)。

步驟

Step 1:先驗證這三個模組「真的沒有」獨立的 `python -m` 前門(grounding-first,不要用猜的)

```
python -m dv_harness.amba_fabric_discovery
echo EXIT:$?
```

這個指令做的事:把 `amba_fabric_discovery.py` 當成腳本執行一次。

真實/預期輸出(已在本環境親自驗證):

```
EXIT:0
```

沒有任何 stdout。原因是這個檔案裡完全沒有 `if __name__ == "__main__":` 區塊——`amba_port_registry.py`、`amba_transaction_ir.py` 也是同樣情形(用 `grep -n "__main__" dv_harness/amba_*.py` 可以自行確認)。這三個模組是純 IR 函式庫,被同專案 `fabric_progress_ir.py` 的 docstring 稱為「real AMBA/SyoSil integration family」,設計上就是給別的模組 import 用,不是拿來當 CLI verb 跑的。

Step 2:用官方真實測試套件,驗證這三個模組「真的可以」被正確呼叫——這其實才是它們真正的 `python -m` 前門

```
python -m pytest dv_harness_tests/test_amba_fabric_discovery.py
dv_harness_tests/test_amba_port_registry.py dv_harness_tests/test_amba_transaction_ir.py -q
```

這個指令做的事:對三個模組各自的官方測試檔案(每份測試都是對著一個真實跑過 `verible-verilog-syntax` 的合成多 master/多 slave AMBA4 SoC fixture,而不是手刻的 Python 物件)跑一次完整驗證。

真實輸出(本 Lab 已在此環境實際執行過):

```
27 passed in 2.36s      # test_amba_fabric_discovery.py
50 passed in 2.57s      # test_amba_port_registry.py
51 passed in 1.35s      # test_amba_transaction_ir.py
```

Step 3:自己寫一個最小的合成 AMBA4 fixture,親手串起「拓模探索 → Port Registry → Transaction IR」

在 scratchpad 建一個 `mini_soc.sv`,內容是一個最小 SoC:一個 AXI4 master(`cpu_core`)→ 一個 1x1 直通式 `axi_fabric` (`S00_AXI_` 進、 `M00_AXI_` 出)→ 一個 AXI4 slave(`ddr_ctrl`)。注意:`cpu↔fabric` 這一段連線、跟 `fabric↔ddr` 這一段連線,一定要用不同的 `wire net` 名稱(下面「常見問題」會說明為什麼)。

再寫一支驅動腳本(因為沒有 CLI,我們直接呼叫真實 API):

```
from pathlib import Path
from dv_harness.amba_fabric_discovery import (
    build_fabric_netlist, build_vip_bind_plan, trace_all_fabric_ports)
from dv_harness.amba_port_registry import (
    build_amba_port_registry, render_amba_port_registry_report)
from dv_harness.amba_transaction_ir import (
    build_transaction_ir_templates, render_amba_transaction_ir_report,
    plan_amba_adapters)
from dv_harness.verible_parser import parse_file

result = parse_file(Path("mini_soc.sv"))
netlist = build_fabric_netlist([result], "soc_top")
traces = trace_all_fabric_ports(netlist, ("u_fabric",))
plan = build_vip_bind_plan(netlist, traces)
rows = build_amba_port_registry(netlist, traces, plan)
templates = build_transaction_ir_templates(rows)
print(render_amba_port_registry_report(rows, traces))
print(render_amba_transaction_ir_report(templates, plan_amba_adapters(rows)))
```

用 `PYTHONPATH=<v50路徑> python run_amba_pipeline.py` 執行。

真實輸出重點(本 Lab 已在此環境親自跑出、逐字節錄):

```
[amba_fabric_discovery] traced 2 fabric ports
  M00_AXI_      dir=TOWARD_DESTINATION_SLAVES status=DESTINATION_FOUND
  S00_AXI_      dir=TOWARD_INITIATING_MASTERS status=SOURCE_FOUND

[amba_port_registry] built 2 AMBA_PORT_REGISTRY rows
...port_id=U_FABRIC_M00_AXI ... endpoint_hierarchy=u_ddr ... trace_status=DESTINATION_FOUND ...
readiness=PARTIAL ... confidence=T2_STRUCTURAL_MATCH
...port_id=U_FABRIC_S00_AXI ... endpoint_hierarchy=u_cpu ... trace_status=SOURCE_FOUND ...
readiness=PARTIAL ... confidence=T2_STRUCTURAL_MATCH

[amba_transaction_ir] built 2 transaction IR templates
port_id=U_FABRIC_M00_AXI protocol=AXI4-Lite Observed Side=SLAVE_SIDE
  Fields Still REQUIRED_HUMAN_INPUT: address, burst_len, ..., transaction_id
```

注意:即使 fabric 端口成功解析出 `DESTINATION_FOUND / SOURCE_FOUND`, `readiness` 仍然只是 `PARTIAL` (`confidence` 只到 `T2_STRUCTURAL_MATCH`), 而且 Transaction IR 每一個 port 都還有一長串 `REQUIRED_HUMAN_INPUT` 欄位 (`clock_domain`、`transaction_id`)。這是刻意設計:拓樸/結構層面能確定的東西(哪個 module、哪個 hierarchy、address/data width)標成 `KNOWN`, 但真正的交易語義(clock domain、transaction id 對應)在「只做 discovery、不做模擬」的階段永遠標成需要人工輸入, 絕不會用猜的填上去。

Step 4:用真實 CLI verb,把手上做出來的 IR 對應到 composite readiness gate

```
python -m dv_harness.cli amba-readiness-gates gates
python -m dv_harness.cli amba-readiness-gates conditions --gate AMBA_PORT_REGISTRY_READY
```

真實輸出(本 Lab 已驗證):

```
L3_REFERENCE_READY
AMBA_PORT_REGISTRY_READY
AMBA_CONSTRAINT_READY
AMBA_CONNECTIVITY_READY
AMBA_VIP_BIND_READY
AMBA_SCOREBOARD_READY
AMBA_COVERAGE_READY
AMBA_TEST_GENERATION_READY
AMBA_SIGNOFF_READY
```

```
Required_Fabric_Ports_Discovered
Required_Roles_Resolved
Required_Hierarchy_Trace_Complete
Required_Clock_Reset_Resolved
Critical_UNKNOWN
```

`amba-readiness-gates` 這個 CLI verb 本身不會去讀你的 RTL 或跑 `build_amba_port_registry()` ——它只吃你自己組好的 `{"condition_name":..., "status":...}` JSON (`--conditions <file>`), 逐一比對這五個條件名稱, 用 worst-wins (單一 `UNMET` 就整個 gate `NOT_READY`) 判斷。這正好呼應 Step 3 的輸出: `Required_Clock_Reset_Resolved` 這一項, 在我們的合成 fixture 上就會是 `UNKNOWN`, 因為 `clock / reset` 欄位在 Port Registry 裡本來就標成 `UNKNOWN`。

驗證

- ☐ `python -m dv_harness.amba_fabric_discovery` 回傳 exit code 0、沒有任何 stdout (確認你理解這是純函式庫, 不是 CLI verb)。
- ☐ 三個模組的 pytest 全數通過, 總數 `27 + 50 + 51 = 128 passed`, 且沒有任何 `SKIPPED` (若有 `SKIPPED`, 代表 `verible-verilog-syntax` 不在 PATH 上)。
- ☐ 你自己寫的 `mini_soc.sv` 跑出的兩個 fabric port (`S00_AXI_ / M00_AXI_`) `trace_status` 都是 `SOURCE_FOUND / DESTINATION_FOUND`, 而不是 `MULTIPLE_SOURCE / MULTIPLE_DESTINATION`; 若不是, 回去檢查 wire net 命名 (見下方常見問題)。
- ☐ 能正確說出: Port Registry 裡 `readiness=PARTIAL`、`confidence=T2_STRUCTURAL_MATCH` 跟 Transaction IR 裡一串 `REQUIRED_HUMAN_INPUT` 欄位, 代表的是「結構解析完成, 但語義尚待人工確認」, 而不是失敗。
- ☐ `amba-readiness-gates conditions --gate AMBA_PORT_REGISTRY_READY` 印出的五個條件名稱, 能對應回你自己在 Port Registry 報表裡實際看到的欄位。

常見問題

1. 「為什麼我的 fixture 跑出 `MULTIPLE_SOURCE / MULTIPLE_DESTINATION` ,而不是乾淨的 `SOURCE_FOUND` ?」這是本 Lab 作者自己第一次寫 `mini_soc.sv` 時真實踩到的錯:如果你把 `cpu↔fabric` 這段連線跟 `fabric↔ddr` 這段連線重複使用同一組 wire net 名稱(例如兩段都叫 `cpu_awvalid`),這兩個邏輯上不同的 AMBA 介面會被合併成同一條 equipotential net, `trace_fabric_port()` 因此誠實地回報「這個 port 有多個候選來源/目的,我不猜」,而不是隨便挑一個當答案。修法:每一段連線各自用獨立的 net prefix(本 Lab 範例用 `cpu_*` 跟 `ddr_*` 區分)。
 2. 「為什麼 `clock_known / reset_known` 永遠是 `UNKNOWN` ?」因為我們寫的合成模組(`cpu_core / ddr_ctrl`)根本沒有任何一個 port 帶 clock 或 reset 命名慣例——AMBA-15 bind location validation 的第 6、7 點會誠實回報 `no port on this instance carries a clock name`。這正對應 CLAUDE.md 的 Bind-Location Rule 3:clock/reset 一定要透過 bind 自己的 port list 明確傳入,絕不能用階層參照(XMR)硬抓;本模組不會猜一個 clock 名字給你。
 3. 「沒有 `verible-verilog-syntax` 的機器上這個 Lab 還能做嗎?」可以,但誠實地說: `fabric_netlist` 這類 fixture 會被 `pytest.mark.skipif` 直接跳過(測試檔自己的說明是「Skipped (never faked) without verible on PATH, the same discipline test_verible_parser.py already follows」),Step 3 的驅動腳本也會在 `parse_file()` 這一步失敗。沒有 verible 的訓練環境裡,請只做 Step 1、Step 2(觀察 SKIPPED 的行數)與 Step 4,並向學員說明:這是 evidence-truth 紀律的一部分——寧可誠實跳過,也不假造一份「解析成功」的輸出。
-

Lab 6: Spec-to-vPlan 與需求可測試性檢查

學習目標

- 理解 spec section 184 的「Canonical Requirement Contract」十五個欄位,以及五值狀態語彙 (COMPLETE / PARTIAL / AMBIGUOUS / CONTRADICTORY / UNKNOWN)如何被 `analyze_requirement_contract()` 從內容重新推導,而不是單純信任 agent 自己填的 `status` 欄位。
- 動手撰寫一份最小、真實符合 `requirement_contract.schema.json` 的需求文件,並透過真實 CLI 與真實 Python API 兩種入口驗證同一份資料。
- 理解 section 218 的「可測試性」(Testability)是與 section 184 的「內容完整性」正交的另一個問題——`requirement_testability.py` 用一組獨立於 `requirement_contract.py` 的三值語彙 (TESTABLE / UNTESTABLE_PROSE / INDETERMINATE)回答「這條需求的文字裡到底有沒有可檢查的條件與可觀測的訊號」。
- 認識 `vplan_artifact.py` (section 184/211 對應的 vPlan 完整性九維度分析)沒有任何 CLI 或 `python -m` 入口——它是一個純 Python API 模組,必須直接 import 使用,並理解為什麼(見「常見問題」)。

前置準備

- 已完成 Lab 1(環境確認、`dv-harness --help` 可正常執行)。
- 工作目錄為 `D:\DV\Task\DV_Agent_Harness_L5\v50` (本 Lab 所有指令皆假設在此目錄下執行)。
- 本 Lab 全程屬於 LOCAL_ANALYSIS(純本地讀檔分析,不碰伺服器、不跑 VCS)——不需要 SSH/Remote Transport Connection Intake。
- 所有暫存檔案請寫到你自己的 scratchpad 目錄,絕不修改 `v50` 專案本身的既有檔案。

步驟

步驟 1:用 `--help` 確認三個模組的真實入口,不要用猜的

先用 CLI 的 `--help` 確認 `requirement-contract` 這個 verb 真實支援的旗標——這是本 Lab 唯一寫進步驟的指令之前,一定要先做的事:

```
python -m dv_harness.cli requirement-contract --help
```

真實輸出:

```
usage: dv-harness requirement-contract [-h] --requirements REQUIREMENTS
                                         [--json] [--fail-on-error]

options:
  -h, --help            show this help message and exit
  --requirements REQUIREMENTS
                        JSON file with a top-level `requirements` list.
                        Records that do not declare `contract_schema_version`
                        are not in this shape and are not analyzed.
  --json                Emit the full machine-readable report instead of the
                        human-readable summary.
  --fail-on-error        Also fail on WARNING findings, not only ERROR.
```

再確認 `requirement_testability.py` 和 `vplan_artifact.py` 各自的真實入口:

```
grep -n "def main\|argparse" dv_harness/requirement_testability.py | head
grep -n "^def main\|argparse" dv_harness/vplan_artifact.py | head
```

你會發現 `requirement_testability.py` 有自己的 `python -m dv_harness.requirement_testability` 入口(但沒有 `dv-harness` 底下的 `verb`),而 `vplan_artifact.py` 完全沒有 `main()` ——它是純 API 模組。這個落差不是文件疏漏, `dv_harness/requirement_contract.py` 模組本身的 `docstring` 就寫明了: `vplan_artifact.py` 這批同一天加入的模組因為 `cli.py` / `gates.py` 當時正被其他並行工作修改,刻意選擇不接 CLI,只留 Python API 或 `python -m` 作為「可達(reached)但未接線(wired)」的入口。

步驟 2:手寫一份最小、真實符合 schema 的需求文件

不要憑印象猜欄位名稱——先看真實 schema 的必要欄位:

```
python -c "
import json
s = json.load(open('dv_harness/schemas/requirement_contract.schema.json'))
print(s['\$defs']['requirement_contract']['required'])
print(s['\$defs']['provenance'])
"
```

真實輸出(節錄):

```
['contract_schema_version', 'requirement_id', 'source', 'feature', 'protocol', 'configuration',
'precondition', 'stimulus', 'expected_result', 'observability', 'checker', 'coverage_intent',
'priority', 'criticality', 'confidence', 'status']
{'type': 'object', 'additionalProperties': False, 'required': ['document'], ...}
```

`source` 是一個 `provenance` 物件,至少要有 `document` 欄位(真實檔案/文件路徑,不能是空字串)。

`priority` / `criticality` / `confidence` 三個列舉值也不要猜,直接從模組讀:

```
python -c "
from dv_harness.requirement_contract import PRIORITY_VALUES, CRITICALITY_VALUES, CONFIDENCE_VALUES
print('priority', PRIORITY_VALUES)
print('criticality', CRITICALITY_VALUES)
print('confidence', CONFIDENCE_VALUES)
"
```

真實輸出:

```
priority ('P0', 'P1', 'P2', 'P3')
criticality ('BLOCKER', 'MAJOR', 'MINOR', 'UNKNOWN')
confidence ('HIGH', 'MEDIUM', 'LOW', 'UNKNOWN')
```

現在把下面內容存成你 `scratchpad` 底下的 `lab6_requirement.json` (十五個欄位全部真實填寫、`source.document` 是一個真實可信的文件名稱、`status` 誠實宣告為 `COMPLETE`):

```
{
  "requirements": [
    {
      "contract_schema_version": "1.0",
      "requirement_id": "REQ-USB3-LINK-001",
```

```

    "source": {
        "document": "usb3_link_training_spec.pdf",
        "locator": "section 7.3.2",
        "quote": "The link partner shall complete Polling.Active within 12ms of receiving TS1 ordered sets."
    },
    "feature": "USB3 link training",
    "protocol": "USB3",
    "configuration": "Gen1x1, SS_PORT0",
    "precondition": "PHY reset deasserted and reference clock stable",
    "stimulus": "Host issues TS1 ordered sets on SS_PORT0 during Polling.LFPS",
    "expected_result": "DUT link state register LTSSM_STATE transitions from Polling.LFPS to Polling.Active within 12ms",
    "observability": "LTSSM_STATE status register, bits [3:0]",
    "checker": "Scoreboard compares observed LTSSM_STATE transition timestamp against 12ms bound",
    "coverage_intent": "Cover Polling.LFPS -> Polling.Active transition under nominal timing",
    "priority": "P1",
    "criticality": "MAJOR",
    "confidence": "HIGH",
    "status": "COMPLETE"
}
]
}

```

步驟 3:透過真實 CLI 執行 requirement-contract

```
python -m dv_harness.cli requirement-contract --requirements <你的 scratchpad 路徑>\lab6_requirement.json
```

真實輸出:

```
requirement-contract: PASS (1 contract-shaped requirement(s) analyzed)
COMPLETE: 1
```

echo \$? 會是 0 (0 = clean)。這代表 `analyze_requirement_contract_set()` 對你手寫的這一條需求跑過每一項檢查(十五個欄位是否都存在、`status` 是否與內容重新推導出的狀態一致、`source` 是否真的有 `document`)全部通過。

步驟 4:直接呼叫 analyze_requirement_contract() 本體,看它真正做了什麼

CLI 的 `requirement-contract` verb 其實是包了一層 `analyze_requirement_contract_set()`,而它對每一條記錄呼叫的正是 `analyze_requirement_contract()`。直接用 Python API 驗證同一份資料,跟 CLI 的結論必須一致:

```
python -c "
import json
from dv_harness.requirement_contract import analyze_requirement_contract, derive_status

record = json.load(open(r'<你的路徑>\lab6_requirement.json', encoding='utf-8'))['requirements'][0]
print('findings:', analyze_requirement_contract(record))
print('derive_status:', derive_status(record))
"
```

真實輸出:

```
findings: []
derive_status: ('COMPLETE', 'every contract field resolved; no open ambiguity or contradiction')
```

`findings` 是空陣列(沒有任何 ERROR/WARNING), `derive_status()` 從內容本身重新推導出 `COMPLETE` ——跟你在 `status` 欄位宣告的值一致,這就是為什麼步驟 3 是乾淨的 PASS。

步驟 5:故意造出一個 `STATUS_OVERCLAIMED` 錯誤,觀察 gate 如何抓到「宣稱完整但內容其實沒填完」

```
python -c "
import json
p = r'<你的路徑>\lab6_requirement.json'
doc = json.load(open(p, encoding='utf-8'))
doc['requirements'][0]['checker'] = 'TBD'
json.dump(doc, open(r'<你的路徑>\lab6_requirement_broken.json', 'w', encoding='utf-8'), indent=2)
"
python -m dv_harness.cli requirement-contract --requirements <你的路徑>\lab6_requirement_broken.json
```

真實輸出:

```
requirement-contract: FAIL (1 contract-shaped requirement(s) analyzed)
COMPLETE: 1
[ERROR] STATUS_OVERCLAIMED (REQ-USB3-LINK-001): status says COMPLETE but the record's own content
derives PARTIAL: unresolved contract field(s): checker
```

`echo $?` 會是 1。注意: `checker` 欄位的內容是 `checker.py` 模組定義的佔位符黑名單(`TBD`、`UNKNOWN`、`N/A`、空字串.....)之一,會被 `is_resolved()` 判定為「沒真正填」,因此就算你的 `status` 還寫著 `COMPLETE`, `derive_status()` 從內容重新推導出的是 `PARTIAL`,兩者不一致 → `STATUS_OVERCLAIMED`。這正是 section 184 存在的核心理由:agent 自己填的 `status` 是一種判斷,不是證據,必須被內容本身反過來檢查。

步驟 6:section 218 的可測試性檢查——一個正交的問題

用你原本乾淨的 `lab6_requirement.json` 跑 `requirement_testability.py` (記住:這個模組沒有 `dv-harness` 底下的 `verb`,只有 `python -m dv_harness.requirement_testability`):

```
python -m dv_harness.requirement_testability --requirements <你的路徑>\lab6_requirement.json
```

真實輸出:

```
requirement-testability: PASS (1 contract-shaped requirement(s) analyzed)
TESTABLE: 1
```

現在把 `expected_result` / `checker` / `observability` 三個欄位全部改成「聽起來已經填完、但完全沒有可檢查條件與可觀測訊號」的空話式文字(例如 "shall function correctly"、"shall behave as expected"),再跑一次:

```
python -m dv_harness.requirement_testability --requirements <你的路徑>\lab6_requirement_prose.json
```

真實輸出:

```
requirement-testability: FAIL (1 contract-shaped requirement(s) analyzed)
UNTESTABLE_PROSE: 1
[WARNING] REQUIREMENT_UNTESTABLE_PROSE (REQ-USB3-LINK-002): requirement reads as untestable prose:
```

```
resolved text carries no checkable condition and no observable signal, only qualitative prose:
checker:'shall behave as expected', expected_result:'shall function correctly',
observability:'adequate'
```

注意這條需求在 `requirement_contract.py` 眼中其實仍然是 schema-COMPLETE(三個欄位都「有填」,不是空字串也不是 TBD),所以步驟 3 那種檢查完全抓不到它——這正是為什麼 section 218 需要一個獨立的三值語彙。

步驟 7: `vplan_artifact.py` ——沒有 CLI,直接寫一支小腳本呼叫 Python API

先看真實模組定義了哪些函式(不要猜函式名或欄位名):

```
grep -n "^def " dv_harness/vplan_artifact.py
```

`build_vplan_hierarchy()` 需要的欄位是 `requirement_refs` / `coverage_refs` / `checker_refs` / `test_refs` (不是 `requirement_ids` / `coverage_ids` 這種直覺猜法——實測時我自己第一次就猜錯了欄位名, `REQUIREMENT_COVERAGE` 維度因此回報成 `PARTIAL` 並列出 `UNMAPPED_REQUIREMENT`,這正是「不要用猜的,先讀原始碼」的活教材)。用正確欄位名寫一支腳本:

```
import json
from dv_harness.vplan_artifact import (
    validate_vplan_document, build_vplan_hierarchy,
    analyze_vplan_completeness, vplan_completeness_report_to_dict,
)

vplan_rows = [
    {"id": "VP-USB3-LINK", "kind": "section", "title": "USB3 Link Training"},
    {
        "id": "VP-USB3-LINK-001", "kind": "item", "parent_id": "VP-USB3-LINK",
        "title": "Polling.LFPS -> Polling.Active within 12ms",
        "requirement_refs": ["REQ-USB3-LINK-001"],
        "verification_method": "SIMULATION",
        "coverage_refs": ["CG-LTSSM-POLLING-ACTIVE"],
        "checker_refs": ["CHK-LTSSM-TIMING"],
        "test_refs": ["TC-USB3-LINK-TRAIN-001"],
        "owner": "usb-dv-team", "priority": "P1",
    },
]
validate_vplan_document(vplan_rows)
build_vplan_hierarchy(vplan_rows)
report = analyze_vplan_completeness(vplan_rows, requirements=[{"id": "REQ-USB3-LINK-001"}])
print(json.dumps(vplan_completeness_report_to_dict(report), indent=2))
```

因為這是純腳本、不是套件內模組,需要把 `v50` 目錄加進 `PYTHONPATH` 才能 `import dv_harness`:

```
set PYTHONPATH=D:\DV\Task\DV_Agent_Harness_L5\v50
python lab6_vplan_demo.py
```

真實輸出(節錄關鍵維度):

```
"REQUIREMENT_COVERAGE": {"status": "READY", "applicable_count": 2, "satisfied_count": 2, "gaps": []},
"COVERAGE_MODEL_LINKAGE": {"status": "READY", ...},
"CHECKER_LINKAGE": {"status": "READY", ...},
```

```
"TEST_STIMULUS_LINKAGE": {"status": "READY", ...},
"INTENT_CROSS_CONSISTENCY": {"status": "UNKNOWN", "applicable_count": 0, "reason": "no
    verification_intents supplied for cross-check (pass verification_intents=[...] to enable
    this dimension)"}
}
```

注意最後一個維度誠實回報 UNKNOWN 而不是假裝 READY ——你沒有傳 verification_intents 參數,模組不會幫你猜一個「乾淨」的結果。

驗證

逐一確認你在本 Lab 中真正得到的是「真實計算出的結果」而不是你以為會發生的結果:

- ☐ python -m dv_harness.cli requirement-contract --help 顯示的旗標只有 --requirements / --json / --fail-on-error,沒有 --vplan 之類你可能以為存在的旗標。
- ☐ 乾淨版 lab6_requirement.json 跑 CLI 得到 PASS、echo \$? 是 0。
- ☐ analyze_requirement_contract() 直接呼叫回傳 findings: [] 且 derive_status() 回傳 ('COMPLETE', ...),與 CLI 結論一致(同一份資料、兩個入口,答案必須相同)。
- ☐ 把 checker 改成 TBD 後重跑,得到 [ERROR] STATUS_OVERCLAIMED、echo \$? 是 1。
- ☐ requirement_testability.py 對同一份乾淨資料回報 TESTABLE: 1;改成空話式 prose 後回報 UNTESTABLE_PROSE: 1 並列出被抓到的確切片語。
- ☐ vplan_artifact.py 的九個維度中,你能講出哪一個維度在沒有提供 verification_intents 時誠實回報 UNKNOWN (而不是 READY),以及為什麼。

常見問題

1. 「我在 dv-harness --help 裡找不到 vplan-artifact 或 requirement-testability 這兩個 verb,是不是我裝壞了?」不是。vplan_artifact.py 和 requirement_testability.py 都是本專案 2026-09-06 同一批新增的模組,CLAUDE.md 對這兩個模組都明確寫著:因為 cli.py / gates.py 當時被同一批工作的其他 agent 並行修改,這兩個模組刻意只留下 Python API(vplan_artifact.py)或 python -m dv_harness.requirement_testability (有自己的 main(),但沒接進 dv-harness 底下)作為入口。CLAUDE.md 用「REACHED(可達)」與「WIRED(已接線)」區分這兩種狀態——這兩個模組都只是 REACHED。跑 grep -n "^def main\|argparse" <module>.py 永遠是判斷一個模組有沒有 CLI 入口最可靠的方法,不要相信文件或記憶。
2. 「為什麼 requirement_contract.py 判定 checker: "TBD" 是錯誤,而不是單純的 WARNING?」因為 TBD (以及 UNKNOWN / N/A / 空字串 / "?" 等,見模組內 _UNRESOLVED_SENTINELS)是模組明確定義的「佔位符」黑名單,is_resolved() 會把它們視為「這個欄位其實沒填」。當 status 宣稱 COMPLETE 但某個欄位其實是佔位符時,derive_status() 從內容重新推導出來的狀態必然更差(本例是 PARTIAL),兩者不一致就是 STATUS_OVERCLAIMED ——這是 SEVERITY_ERROR,因為模組文件裡明講:「a COMPLETE requirement is the only thing downstream_consumable() lets a generator build from」,宣稱完整但其實沒填完,會讓下游 generator 誤信一份不完整的需求。反過來,宣稱比內容「更差」的狀態(例如內容其實已達 COMPLETE,但 status 保守寫成 PARTIAL)只會得到 WARNING,不是 ERROR。
3. 「requirement_testability.py 判定 UNTESTABLE_PROSE 的規則是不是很聰明的語意分析?」不是,模組自己的 docstring 講得很清楚:這是「heuristic, stated as such」的正則表達式模式掃描(比較運算子、十六進位/Verilog 數字字面量、assert / scoreboard 等關鍵詞、SIGNAL_LIKE 的全大寫底線命名 token.....),不是對需求語意的真正理解。用模組列出的固定片語清單(shall function correctly、be robust.....)之外的空話寫法,模組完全抓不到,只會誠實回報 INDETERMINATE (而不是錯誤地判為 TESTABLE)。這是刻意的設計取舍:模組文件明講「a false TESTABLE (missing a real defect) is a worse failure mode than an honest INDETERMINATE」,所以偵測規則寫得偏窄而不是偏廣。

Lab 7: 模擬結果解析與 LSF Regression 概念(無真實伺服器時的替代路徑)

學習目標

- 學會使用 `dv-harness sim-log-analyze` 對一份 `sim.log` 做離線、唯讀的失敗特徵(signature)解析,並讀懂它輸出的 `epilogue` 與 `signatures` 區塊。
- 理解本專案文件化的 **FINAL CHECK epilogue** 格式(`FINAL CHECK @ <time> ns / UVM_FATAL = N, UVM_ERROR = N, UVM_WARNING = N / VERDICT: PASSED|FAILED`),並能自己寫出一份符合這個格式的合成(synthetic)`sim.log` 來練習。
- 理解 `preflight.py` (`dv-harness preflight`) 在「有無真實 LSF/License Server」兩種情境下分別會回傳什麼,並能正確解讀 `sandbox` 環境中的 `BLOCKED` 結果,而不是誤以為它壞掉了。
- 認識 `regression_reporter.py` (`dv-harness lsf-watch-start / lsf-watch-status`) 在真實 LSF regression 監控中扮演的角色,以及為什麼這一段在本訓練環境中只能讀不能寫(唯讀檢查安全,實際啟動需要真實帳號與伺服器)。

前置準備

- 已完成前面幾個 Lab,熟悉 `python -m dv_harness.cli --help` 的用法與 `dv-harness` 對應的是 `python -m dv_harness.cli`。
- 本機已可執行 `python -m dv_harness.cli <verb> --help` (不需要專案根目錄以外的權限)。
- 執行模式宣告(Execution Mode Gate):**本 Lab 全程屬於 `LOCAL_ANALYSIS` ——「這個是純本地讀檔分析(不碰伺服器、不跑 VCS)」,凡是需要真正連上 LSF 伺服器/License Server 才能得到 `PASS` 的步驟,本 Lab 會明確標示「此步驟需要真實 LSF farm 連線」,並直接告訴你在沒有伺服器的訓練環境中應該預期看到什麼樣的真實輸出,絕不假造一個看起來像跑過真實 regression 的結果。
- 準備一個可寫入的暫存目錄,用來放合成的 `sim.log` fixture(本 Lab 一律使用 `scratch` 目錄,不會動到 `v50` 專案本身的任何檔案)。

步驟

Step 1: 確認 `sim-log-analyze` 的真實旗標

```
python -m dv_harness.cli sim-log-analyze --help
```

真實輸出(已在本地驗證):

```
usage: dv-harness sim-log-analyze [-h] (--log LOG | --log-text LOG_TEXT)

options:
  -h, --help            show this help message and exit
  --log LOG              Path to a sim.log file.
  --log-text LOG_TEXT   Inline log text (for testing).
```

這是這個 verb 唯一支援的兩個互斥旗標——直接讀檔或直接餵字串,兩者只能擇一。這一步只是核對真實旗標,不做任何修改。

Step 2: 撰寫一份符合本專案 **FINAL CHECK epilogue** 格式的合成 `sim.log`

CLAUDE.md 與 `dv_harness/sim_log_analysis.py` 明確定義了這個專案「真實文件化」的 epilogue 格式(見 `sim_log_analysis.py` 第 325–338 行的註解, `parse_epilogue()` 的規則):

```
FINAL CHECK @ <time> ns
UVM_FATAL = N, UVM_ERROR = N, UVM_WARNING = N
VERDICT: PASSED|FAILED
```

在你的 scratch 目錄下建立 `sim_sample.log`, 內容如下(這是本 Lab 自行撰寫的合成 fixture, 刻意包含一個 `sim_log_analysis.py` 文件中點名的真實陷阱: 一行沒有 `UVM_` 前綴、純文字的 `Error-[XYZ]`):

```
UVM_INFO @ 100: reporter [RNTST] Running test usb3_link_train_test...
UVM_INFO @ 5000: uvm_test_top.env.agent0.monitor [MON] Packet received: LFPS detected
Error-[XYZ] PLL model in usb3_phy did not lock within timeout
UVM_ERROR @ 12000: uvm_test_top.env.scoreboard [SCB] Data mismatch: expected 0xDEAD, got 0xBEEF
UVM_INFO @ 20000: uvm_test_top.env.agent1.driver [DRV] Sending packet 42
FINAL CHECK @ 25000 ns
UVM_FATAL = 0, UVM_ERROR = 1, UVM_WARNING = 3
VERDICT: FAILED
```

這一步只是本地寫檔案, 不牽涉任何伺服器。

Step 3: 對這份合成 `sim.log` 執行真實的離線解析

```
python -m dv_harness.cli sim-log-analyze --log <你的 sim_sample.log 完整路徑>
```

這個指令做的事: 純本地讀檔, 用 `sim_log_analysis.parse_sim_log_file()` 掃描全文中的 `UVM_FATAL` / `UVM_ERROR` / bare `Error-` 等標記, 並嘗試抽出文件化的 FINAL CHECK epilogue 三行。真實輸出(已在本地跑過, 逐字節錄):

```
{
  "total_lines": 8,
  "epilogue": {
    "final_check_time_ns": 25000.0,
    "uvm_fatal": 0,
    "uvm_error": 1,
    "uvm_warning": 3,
    "verdict": "FAILED"
  },
  "signatures": [
    {
      "signature": "UVM_ERROR @ <N>: uvm_test_top.env.scoreboard [SCB] Data mismatch: expected
        0x<HEX>, got 0x<HEX>",
      "category": "scoreboard_mismatch",
      "severity": "HIGH",
      "count": 1,
      "first_line_no": 4,
      "last_line_no": 4,
      "example_line": "UVM_ERROR @ 12000: uvm_test_top.env.scoreboard [SCB] Data mismatch: expected
        0xDEAD, got 0xBEEF",
      "markers": ["UVM_ERROR", "MISMATCH", "SCOREBOARD"]
    },
    {
      "signature": "Error-[XYZ] PLL model in usb3_phy did not lock within timeout",
      "category": "timeout",
```

```

    "severity": "MEDIUM",
    "count": 1,
    "first_line_no": 3,
    "last_line_no": 3,
    "example_line": "Error-[XYZ] PLL model in usb3_phy did not lock within timeout",
    "markers": ["BARE_ERROR", "TIMEOUT"]
}
]
}

```

請注意兩件事:(1) `epilogue.verdict` 正確讀到 "FAILED", 且三個數字(fatal/error/warning)與你寫進去的合成資料完全一致;(2) 那一行沒有 `UVM_` 前綴的 `Error-[XYZ]` ... 真的被抓出來了, `markers` 裡有 `BARE_ERROR` ——這正是這個模組文件裡點名的真實案例:某個 PLL model 曾經印出一行純文字 `ERROR in PLL model`, 被一個只認 `UVM_ERROR` 的舊 filter 漏掉將近一小時, `parse_sim_log()` 因此被設計成永遠額外掃描裸 `Error- / ERROR` 字樣, 不只認 `UVM_` 詞彙表。

Step 4:執行 `preflight`, 並誠實理解 `sandbox` 中的結果

```
python -m dv_harness.cli preflight
```

這個指令做的事:真正呼叫 `preflight.py` (`dv_harness/preflight.py`) 依序檢查 EDA license 是否可用(`lmstat`)、LSF queue 健康度(`bqueues`)、主機可達性、磁碟空間、`workdir`、以及 `VCS_HOME / UVM_HOME / VERDI_HOME` 環境變數——這一步需要一台真正連得到 License Server 與 LSF Scheduler 的機器才能得到全部 **PASS**。在本訓練沙盒(沒有設定 `preflight.license_server`、也沒有 `bqueues / lmstat` 可執行檔)中,你應該預期看到、也確實會看到 **BLOCKED**, 真實輸出如下:

```

{
  "overall": "BLOCKED",
  "blocked_on": ["eda_license", "lsf_queue_health", "eda_env_vars"],
  "checks": [
    {"name": "eda_license", "status": "FAIL",
     "detail": "No license server configured (PreflightConfig.license_server is empty) -- cannot verify EDA license availability before submission."},
    {"name": "lsf_queue_health", "status": "FAIL",
     "detail": "bqueues failed: exit_code=1", "command": "bqueues vcs"},
    {"name": "host_reachability", "status": "PASS",
     "detail": "target host reachable (hostname=PETER-LIN)"},
    {"name": "disk_space", "status": "SKIP",
     "detail": "no workdir configured; disk-space check skipped."},
    {"name": "workdir", "status": "SKIP",
     "detail": "no workdir configured; workdir check skipped."},
    {"name": "eda_env_vars", "status": "FAIL",
     "detail": "env-var check command failed: exit_code=1"}
  ]
}

```

這不是失敗——這是 `preflight.py` 在「誠實回報缺少的先決條件」, 而不是假裝一切正常。若要在真實環境重跑並拿到 **PASS**, 你需要用 `--license-server 2900@host-a` 這類旗標指到一台真的 License Server, 並在一台裝有 `bqueues` (LSF) 的 Linux 主機上執行——這正是本專案 `CLAUDE.md` 的 SSH/Remote Transport Connection Intake 規則存在的原因:連上真正的 Linux DV Server 前, 必須先詢問使用者是否要建立連線, 絕不能在對話裡自行嘗試連線或印出密碼。

Step 5(唯讀, 示範 `regression_reporter.py` 的角色):確認目前沒有背景 LSF watcher 在跑

```
python -m dv_harness.cli lsf-watch-status
```

真實輸出:

```
{"running": false, "pid": null}
```

`lsf-watch-status` 是唯一一個在本 Lab 沙盒中可以安全執行、且真正回傳有意義結果的 LSF 相關指令,因為它只讀一個本地 PID 檔案,不連任何伺服器。它背後對應的 `dv-harness lsf-watch-start --vcuser <account>` 才是真正啟動 `regression_reporter.py` 背景監控迴圈的指令(真實旗標如下,僅供核對,本 Lab 不執行,因為它需要真實 LSF 帳號才有意義):

```
usage: dv-harness lsf-watch-start [-h] --vcuser VCUSER
                                   [--uvm-root-path UVM_ROOT_PATH]
                                   [--interval-minutes INTERVAL_MINUTES]
```

`regression_reporter.py` 的真實運作方式,是每隔 `--interval-minutes` (預設 5 分鐘, `DEFAULT_INTERVAL_MINUTES`) 跑一次 reconciliation cycle:把每個已知的 LSF `JobState` 拿去跟真正的 `bjobs /log` 狀態核對,同時維護 `regression.list` 這個安全網。`CLAUDE.md` 規定這個間隔的上限是 10 分鐘(`SPEC_MAX_INTERVAL_MINUTES`, 對應 `self_check_list.md` #41 的要求),慢於 5 分鐘仍合法,但 `lsf-watch-start` 會在 `stderr` 印出明確警告,絕不默默降頻。

驗證

- Step 3 的 JSON 輸出中, `epilogue.verdict` 必須等於 `"FAILED"`,且 `uvm_fatal` / `uvm_error` / `uvm_warning` 三個數字與你寫進 fixture 的完全一致——代表 `parse_epilogue()` 真的抓到了你自己寫的 FINAL CHECK 三行,而不是猜出來的。
- Step 3 的 `signatures` 陣列中,第二筆 `markers` 必須包含 `"BARE_ERROR"` ——代表你確認了這個模組真的會去抓沒有 `UVM_` 前綴的裸錯誤行,不是只認 UVM 詞彙表。
- Step 4 的輸出 `overall` 必須是 `"BLOCKED"` (在沒有設定真實 license server / 沒有 `bqueues` 的環境中),且 `blocked_on` 至少要包含 `"eda_license"` 與 `"lsf_queue_health"` ——如果你在自己的機器上跑出 `"PASS"`,代表你的機器其實已經配好了真實的 EDA 工具鏈,這也是合理、可預期的結果,只是跟本文件描述的沙盒情境不同。
- Step 5 的輸出必須是 `{"running": false, "pid": null}` (假設你先前沒有在這台機器上啟動過 watcher)。

常見問題

- 「為什麼 `sim-log-analyze` 抓不到我 `sim.log` 裡的某個奇怪錯誤格式?」這是預期行為,不是 bug。`sim_log_analysis.py` 的 marker 詞彙表是固定的一組 `regex(UVM_FATAL / UVM_ERROR / UVM_WARNING / bare Error- / Fatal / assertion / timeout / mismatch / scoreboard / protocol / license / killed / memory / crash)`,來自 `.claude/skills/CORE/failure-triage/SKILL.md` 的文件化詞彙。一行文字若不匹配任何一個 marker,就不會出現在 `signatures` 裡——這正是為什麼 Step 3 特別示範了「裸 `Error-` 也會被抓到」這個真實修正過的陷阱,但這不代表它能抓到所有可能的錯誤格式。
- 「preflight 在我的 Windows 機器上為什麼連 `bqueues` 都找不到?」這是真實、預期的環境限制,不是這個模組壞掉。`bqueues` / `bsub` / `lmstat` 都是 Linux LSF/License 工具鏈上才有的執行檔,在一台單純的 Windows PC 上原生不存在。`CLAUDE.md` 的「Remote Linux Execution (Persistent Relay)」章節明確說明:要在真正的 Linux DV Server 上跑 `preflight --remote`,必須先透過 SSH/Remote Transport Connection Intake Gate 詢問使用者是否建立連線,再用 `tools/remote/remote_exec.py` 下指

令; `remote_relay.py` 本身永遠不能被 Claude Code 的工具呼叫直接執行(即使不在指令文字裡打密碼也不行),這是本專案 2026-08-31 一次真實事故後定下的鐵律。

3. 「我可以在沒有真實 LSF 帳號的情況下練習 `lsf-watch-start` 嗎?」不行,也不建議假裝可以。 `--vcuser` 是必要旗標,且這個指令的整個存在意義就是去發現「這個帳號底下正在跑的真實 LSF job」——沒有真實帳號與真實伺服器連線,這個指令沒有任何有意義的東西可以做。這正是本 Lab 只示範唯讀的 `lsf-watch-status` (回報「目前沒有在跑」)、而不去執行 `lsf-watch-start` 本身的原因:與其在訓練環境裡編造一個看起來成功的假輸出,不如誠實停在「這一步需要真實伺服器連線」這個邊界上。
-

Lab 8: Coverage 結案、Waiver 與 Golden Scenario 上機練習

學習目標

- 理解 `golden_scenario.py` 如何把「這次 PASS 是針對哪個 commit 驗證過的」變成一顆可查詢、可即時判斷新鮮度(FRESH/STALE)的 capsule,而不是一個寫死不會過期的旗標。
- 理解 `waiver_store.py` 的五值狀態機(VALID / REVALIDATION_REQUIRED / EXPIRED / REVOKED / UNKNOWN)是「每次讀取都即時推導」出來的,而不是存在檔案裡的固定欄位。
- 動手跑 `functional_coverage_signoff.py` 的 Closure 公式: $\text{Closure \%} = 100 * (\text{Covered_bins} + \text{ApprovedWaiver_bins} + \text{ProvenUnreachable_bins}) / \text{Total_declared_goal_bins}$,親眼看到 waiver 如何把一個沒打到的 coverage bin 算進結案百分比。
- 練習在一個全新的 scratch 專案(絕不動 `v50` 正式專案本身)裡,用真正的 git repo、真正的 DuckDB evidence store、真正的 waiver ledger,把這三個模組串起來跑一次完整流程。

前置準備

- 已在 `v50` 專案根目錄下, `python -m dv_harness.cli --help` 能正常執行(本 lab 已用這個指令實際確認過 `golden-scenario`、`waiver-store`、`functional-coverage-signoff` 三個 verb 都真的存在於目前的 CLI)。
- 這整個 lab 屬於「這是純本地讀檔分析(不碰伺服器、不跑 VCS)」的 LOCAL_ANALYSIS 模式:不需要連 LSF、不需要 VCS/Verdi、不需要任何 SSH 連線。
- 系統上需要有 `git` 可執行——`golden_scenario.py` 的新鮮度判定是真的呼叫 `git diff` 算出來的,不是模擬的。
- Python 環境需已安裝 `duckdb` (`evidence.duckdb` 走的是 DuckDB)。
- 準備一個全新的 scratch 目錄(例如你自己的暫存資料夾),本 lab 會在裡面初始化 git repo、寫入 DuckDB 檔案、寫入 waiver ledger——絕對不要指向 `v50` 專案本身。

步驟

Step 1:先用 `--help` 確認三個模組的真實 CLI 語法(不要用猜的)

```
python -m dv_harness.cli golden-scenario --help
python -m dv_harness.cli waiver-store --help
python -m dv_harness.cli functional-coverage-signoff --help
```

這三個指令分別列出三個 verb 各自真正接受的 flag。實際跑出來的內容是:

```
usage: dv-harness golden-scenario [-h] [--json-file JSON_FILE]
                                   [--capsule-id CAPSULE_ID] [--head HEAD]
                                   [--db DB] [--vip-version TOOL=VERSION]
                                   [--json]
                                   {record,list,status}
```

```
usage: dv-harness waiver-store [-h] [--json] {statuses,list,status}
```

```
usage: dv-harness functional-coverage-signoff [-h] [--json]
```

注意 `functional-coverage-signoff` 只有 `record / list / status` 這種子指令是沒有的——它是一個「唯讀 `rollup`」,只有一個結果,沒有子動作。稍後會回頭講這個 `verb` 為何值得注意(見「常見問題」第 1 點)。

Step 2:建立一個 scratch 專案:真的 git repo + 真的一筆 PASS 證據

`golden_scenario.record_golden_scenario()` 會硬性要求 `evidence_id` 必須真的存在於 `evidence.duckdb` 的 `normalized_evidence` 表裡,而且該筆證據必須是「真的 PASS」——不是憑空造一個 JSON 就能餵進去。所以第一步要先用 `vip_distill.distill_sim_log()` 對一份真的 `sim.log` 做真的萃取,再用 `evidence_db.EvidenceStore` 寫入。存成 `setup.py` 執行:

```
# setup.py -- 在 scratch 目錄裡建 git repo + 真的 evidence.duckdb
import subprocess, sys
from pathlib import Path
sys.path.insert(0, r"D:\DV\Task\DV_Agent_Harness_L5\v50")
from dv_harness.evidence_db import EvidenceStore
from dv_harness.lsf_client import JobState
from dv_harness.vip_distill import distill_sim_log

ROOT = Path(sys.argv[1])
PASSING_LOG = """\
UVM_INFO @ 1200 ns: uvm_test_top.env.usb3_agent [LFPS] link training complete
FINAL CHECK @ 25000 ns
UVM_FATAL = 0, UVM_ERROR = 0, UVM_WARNING = 0
VERDICT: PASSED
"""

(ROOT / "rtl" / "usb3_link").mkdir(parents=True, exist_ok=True)
(ROOT / "rtl/usb3_link/usb3_link_ctrl.v").write_text(
    "module usb3_link_ctrl(input clk, output reg lfps_done);\nendmodule\n")
(ROOT / ".gitignore").write_text(".dv-harness/\nrun/\n")
subprocess.run(["git", "init", "-q", "-b", "master", str(ROOT)], check=True)
subprocess.run(["git", "-C", str(ROOT), "config", "user.email", "lab8@x.invalid"])
subprocess.run(["git", "-C", str(ROOT), "config", "user.name", "lab8"])
subprocess.run(["git", "-C", str(ROOT), "add", "-A"], check=True)
subprocess.run(["git", "-C", str(ROOT), "commit", "-qm", "init"], check=True)
sha = subprocess.run(["git", "-C", str(ROOT), "rev-parse", "HEAD"],
    capture_output=True, text=True).stdout.strip()

log = ROOT / "run/555001/sim.log"
log.parent.mkdir(parents=True, exist_ok=True)
log.write_text(PASSING_LOG)
store = EvidenceStore(ROOT / ".dv-harness/evidence/evidence.duckdb")
env = distill_sim_log(log_path=log, job_id=555001, pattern="usb3_lfps_basic",
    protocol="USB3", run_dir=str(log.parent))
store.insert_normalized_evidence(env)
store.insert_job_state(JobState(job_id=555001, regression_id="REG-1",
    pattern="usb3_lfps_basic", run_dir=str(log.parent), sim_log=str(log),
    seed="42", lsf_status="DONE", sim_status="PASS",
    uvm_error_count=0, uvm_fatal_count=0, git_sha=sha))
store.insert_coverage_sample(
    {"name": "usb3_lfps_states", "percent": 100.0, "bins_total": 4, "bins_hit": 4},
    source="scratch/summary.json")
store.insert_coverage_sample(
    {"name": "usb3_link_speeds", "percent": 50.0, "bins_total": 4, "bins_hit": 2},
    source="scratch/summary.json")
```

```
store.close()
print("EVIDENCE_ID=" + env["evidence_id"]); print("SHA=" + sha)
```

```
python setup.py <你的scratch路徑>
```

實際跑出來的真實輸出(每次跑 evidence_id/sha 都不同,這是內容雜湊出來的):

```
EVIDENCE_ID=EVID-6f37c1a1b3eaf0bbbd058e6
SHA=8c9bdba4025ca0ff746301b01bcca157976c1ace
```

把這兩個值記下來,後面步驟要用。

Step 3:用 `dv-harness golden-scenario record` 記錄一顆真的 golden capsule

先寫一個 `capsule.json` (`evidence_id` 換成你自己 Step 2 印出來的值):

```
{
  "capsule_id": "GS-lab8.usb3_link.usb3_lfps_basic",
  "project": "lab8",
  "subsystem": "usb3_link",
  "test_name": "usb3_lfps_basic",
  "evidence_id": "EVID-6f37c1a1b3eaf0bbbd058e6",
  "protocol": "USB3",
  "watched_paths": ["rtl/usb3_link"]
}
```

```
python -m dv_harness.cli golden-scenario record \
  --json-file <scratch>/capsule.json \
  --db <scratch>/dv-harness/evidence/evidence.duckdb
```

實際輸出:

```
recorded golden scenario GS-lab8.usb3_link.usb3_lfps_basic (test=usb3_lfps_basic,
verified_sha=8c9bdba4025ca0ff746301b01bcca157976c1ace, evidence_id=EVID-6f37c1a1b3eaf0bbbd058e6)
```

`watched_paths` 就是這顆 capsule 「盯著」的目錄——之後只要 `rtl/usb3_link` 底下有真的 RTL 變更,這顆 capsule 就會過期。

Step 4:用 `golden-scenario status` 驗證新鮮度,然後做一個真的 commit 讓它過期

```
python -m dv_harness.cli --project-root <scratch> golden-scenario status \
  --db <scratch>/dv-harness/evidence/evidence.duckdb
```

```
GOLDEN SCENARIO FRESHNESS: FRESH (head=HEAD, capsules=1)
FRESH=1 STALE=0 UNKNOWN=0

[FRESH] GS-lab8.usb3_link.usb3_lfps_basic
...
- NO_BEHAVIORAL_CHANGE_IN_SCOPE_SINCE 8c9bdba4025c (0 file(s) changed ...)
```

現在真的改一行 RTL 並 commit:

```
echo "// widened lfps timeout window" >> <scratch>/rtl/usb3_link/usb3_link_ctrl.v
git -C <scratch> add -A && git -C <scratch> commit -qm "widen lfps timeout window"
```

再跑一次同一個 `status` 指令,這次是真的變 STALE、退出碼變成 1:

```
GOLDEN SCENARIO FRESHNESS: STALE (head=HEAD, capsules=1)
  FRESH=0  STALE=1  UNKNOWN=0

[STALE] GS-lab8.usb3_link.usb3_lfps_basic
  - DESIGN_OR_CONFIG_CHANGED_SINCE_VERIFIED_SHA: rtl/usb3_link/usb3_link_ctrl.v (HIGH)
```

Step 5:用 `waiver-store` 先看誠實的「空」,再記錄一筆真的 waiver

```
python -m dv_harness.cli --project-root <scratch> waiver-store status
```

```
{
  "status": "NOT_AVAILABLE",
  "reason": "NO_WAIVER_STORE",
  "path": "...\\dv-harness\\waivers\\waivers.json",
  "waivers": []
}
```

(退出碼 2——「查不到」跟「查了是乾淨的」是兩種不同答案,這個模組不會把兩者混為一談。)

用 `waiver_store.record_waiver()` 記一筆真的 waiver(`usb3_link_speeds` 這個 bin 有兩格沒打到):

```
from dv_harness import waiver_store as ws
from pathlib import Path
ws.record_waiver(Path("<scratch>"), {
    "waiver_id": "WAIVER-LAB8-1", "item": "usb3_link_speeds",
    "reason": "Gen2 speed bin unreachable on this board rev.",
    "evidence": "doc/notes.md#board-strap",
    "scope": {"requirement_ids": ["REQ-USB3-SPEED-1"], "subsystem": "usb3_link",
              "spec_revision": "rev-A", "design_evidence_hash": "sha256:deadbeef",
              "approval_id": "APPROVAL-LAB8-1", "scope_hash": "sha256:cafef00d"},
    "approver": "lab8-reviewer", "affected_version": {"rtl_hash": "sha256:0000"},
    "risk": "LOW", "created_at": "2026-09-07T00:00:00Z",
    "expires_at": "2099-01-01T00:00:00Z",
})
```

再跑一次 `waiver-store status`,這次是真的 VALID:

```
{
  "status": "CLEAR",
  "waivers": [{"waiver_id": "WAIVER-LAB8-1", "item": "usb3_link_speeds",
                "status": "VALID", "status_reason": "WITHIN_EXPIRY_AND_NO_TRIGGER_FIRED", ...}],
  "not_valid": 0
}
```

Step 6:用 `functional-coverage-signoff` 跑一次真的 Closure 公式

```
python -m dv_harness.cli --project-root <scratch> functional-coverage-signoff
```

```
# FUNCTIONAL COVERAGE SIGNOFF

**SIGNOFF_READY** -- FUNCTIONAL_COVERAGE_SIGNOFF_READY=True, Closure=100.0%

Waiver ledger: AVAILABLE
Evidence database: AVAILABLE
Testplan correspondence: NOT_AVAILABLE (NO_ENV_MANIFEST)
Declared coverage-goal scope: ALL_RECORDED_CATEGORIES (no testplan correspondence was available ...)

| Coverage Bin | Percent | Bins Missing | Classification | Credit | Waiver |
|---|---|---|---|---|---|
| usb3_lfps_states | 100.0% | 0 | - | MEASURED_COVERED | - |
| usb3_link_speeds | 50.0% | 2 | MISSING_TEST | APPROVED_WAIVER | VALID |
```

注意 `usb3_link_speeds` 明明只實測到 50%,但因為有一筆 `VALID` 的 `waiver` 頂著, `Credit` 欄變成 `APPROVED_WAIVER`, 整體 `Closure` 就衝到 100%。這正是這個 lab 前面說的公式在真實跑。因為這個 `scratch` 專案沒有 `env.manifest.json`, `Testplan correspondence` 誠實顯示 `NOT_AVAILABLE`, 並把「宣告的目標範圍」自動放寬成「evidence DB 裡曾經記過的所有分類」——這也是誠實揭露,不是假裝乾淨。

Step 7(驗證撤銷 waiver 的行為):把 waiver 撤銷,看 Closure 怎麼崩

```
ws.revoke_waiver(Path("<scratch>"), "WAIVER-LAB8-1",
                 revoked_by="lab8-reviewer", reason="board rev fixed, Gen2 now reachable")
```

```
python -m dv_harness.cli --project-root <scratch> functional-coverage-signoff
```

```
**BLOCKED_BY_WAIVER** -- FUNCTIONAL_COVERAGE_SIGNOFF_READY=False, Closure=75.0%
```

一撤銷, `usb3_link_speeds` 那 2 個 bin 立刻不再被計入 `Closure`(掉到 75%),而且只要專案裡任何一筆 `waiver` 是 `REVOKED/EXPIRED/UNKNOWN`,整個 `signoff` 就被擋住——即使 `Closure` 本身已經算到 100%,也一樣過不了。

驗證

- ☐ Step 1 的三個 `--help` 都能正常印出,且你有看到 `golden-scenario` 真的有 `record/list/status` 三個子動作, `waiver-store` 有 `statuses/list/status`, `functional-coverage-signoff` 沒有子動作。
- ☐ Step 3 的 `golden-scenario record` 指令退出碼是 0,且輸出訊息裡的 `evidence_id` 跟你 Step 2 印出來的值完全一致。
- ☐ Step 4 兩次 `status` 分別看到 `FRESH` (退出碼 0)與做完真的 `commit` 後的 `STALE` (退出碼 1),且 `STALE` 那則發現有明確點名是哪個檔案(`rtl/usb3_link/usb3_link_ctrl.v`)觸發的。
- ☐ Step 6 的 `Closure` 百分比、`Credit` 欄位跟本 lab 貼出的一致(`usb3_link_speeds` 顯示 `APPROVED_WAIVER`)。
- ☐ Step 7 撤銷 `waiver` 後, `Closure` 百分比真的下降,且整體狀態變成 `BLOCKED_BY_WAIVER`。

常見問題

1. `CLAUDE.md` 上寫的跟目前 CLI 實際狀況有落差時,以誰為準? 本專案自己在 `CLAUDE.md` 「Evidence Truth Rule」裡就明講:「CLAUDE.md is project guidance, not verification evidence... If CLAUDE.md

conflicts with current evidence, current evidence wins」。實際上 `CLAUDE.md` 裡「Functional Coverage Signoff Rollup」那段寫的是「no `cli.py` verb yet -- `python -m dv_harness.functional_coverage_signoff` is the only front door」,但本 lab Step 1 用 `python -m dv_harness.cli functional-coverage-signoff --help` 實際跑出來卻證明這個 verb 現在已經存在於 `dv-harness` CLI 裡。這正是本專案自己要求的紀律:任何一句文件描述,動手跑一次當下的 `--help` 或指令去驗證,永遠比相信文件裡寫的日期更可靠。

2. 為什麼 `record_golden_scenario()` 會直接丟例外而不是「盡量記一筆看起來合理的資料」? 因為它硬性要求 `evidence_id` 必須真的存在於 `normalized_evidence` 表、必須是真的 PASS、`test_name` 必須跟該筆證據的 `pattern` 一致——這三個檢查任何一個沒過都會拋出 `EvidenceNotFoundError` / `EvidenceNotPassingError` / `EvidenceMismatchError`。如果你在 Step 3 貼錯 `evidence_id` (例如複製到舊的一次跑法印出來的值),會直接看到明確的錯誤訊息而不是一顆看似正常、實際上指向錯誤證據的假 capsule。
 3. `REVALIDATION_REQUIRED` 狀態的 `waiver` 會不會擋 `signoff`? 不會,也不算入 Closure——`functional_coverage_signoff.py` 的公式裡明講「A `REVALIDATION_REQUIRED` waiver neither counts toward closure(只有 `VALID` 才算)nor blocks signoff(只有 `EXPIRED` / `REVOKED` / `UNKNOWN` 才擋)」。這跟直覺「非 `VALID` 狀態應該都算有問題」不一樣,是本模組刻意的設計:一筆等著覆核但還沒到期的 `waiver`,既不能拿來充當已結案的證據,但也還沒到「明確有問題」需要擋人的程度。
-

Lab 9: Loop Engineering、AI 推理與人機互動(question_queue)實作

學習目標

- 學會使用 `dv-harness question-queue` 的 3-tier(Tier-1 自我解決 / Tier-2 安全假設 / Tier-3 必須詢問人類)ask/answer 協定,實際跑一次完整的「提問 → 人類回答 → 決策落地」流程。
- 理解 `question_queue.py` 如何用 `classify_tier()` 對每個問題即時分級,並觀察 Tier-2(ASSUMED)與 Tier-3(OPEN → ANSWERED)在實際輸出上的差異。
- 學會用 `python -m dv_harness.loop_telemetry` 與 `dv-harness loop-budget` 這兩個 module-level 前門,檢視 section 107/108(Loop Engineering Center 事件與資料表)與 section 91-93(統一 budget engine / 失敗分類 / circuit breaker)的真實資料。
- 建立「這些工具在一個全新、沒有任何 loop 執行過的 scratch 專案上會誠實回報什麼」的直覺——也就是本專案反覆強調的 Evidence Truth Rule:沒有證據就回報沒有證據,絕不假造。

前置準備

- 已在本機安裝好 `D:\DV\Task\DV_Agent_Harness_L5\v50` 這個 checkout,且可以用 `python -m dv_harness.cli --help` 正常執行(代表 Python 環境與 `dv_harness` package 都已就緒)。
- 本 Lab 全程屬於 LOCAL_ANALYSIS(純本地讀檔/寫檔分析,不連任何伺服器、不跑 VCS/LSF)——依照 CLAUDE.md 的 Execution Mode Gate,這點必須先聲明清楚。
- 準備一個乾淨的 scratch 目錄(本文以 `/tmp/lab9_scratch` 為例;Windows 使用者可換成 `C:\Users\<you>\AppData\Local\Temp\lab9_scratch` 之類的暫存路徑),絕對不要對著 v50 專案本身的 `.dv-harness/` 跑寫入型指令——所有指令都要帶 `--project-root <scratch>`。

步驟

Step 1:先用 `--help` 確認這三個子系統的真实 CLI 介面(不要憑記憶猜測旗標)

```
python -m dv_harness.cli --help
```

輸出的頂層 subcommand 清單很長,但可以在其中找到 `question-queue`、`loop-budget`、`loop-telemetry` (`loop_telemetry` 本身沒有 `dv-harness` CLI verb,只有 `python -m dv_harness.loop_telemetry` 這個獨立前門,下面會示範)。這一步的重點是:清單本身就是 ground truth,任何舊文件裡列的旗標名稱如果跟這裡對不上,一律以這裡為準。

Step 2:查看 `question-queue` 的真實子指令與 `add / answer` 的完整參數

```
python -m dv_harness.cli question-queue --help
python -m dv_harness.cli question-queue add --help
python -m dv_harness.cli question-queue answer --help
```

真實輸出會看到 `add` 需要的必要旗標是: `--domain {vip,dut,env}`、`--question`、`--context-path`、`--option` (可重複給 2-3 個)、`--recommendation` (必須等於某個 `--option` 的值)、`--assumption-if-unanswered`;另外還有三個決定「是否為 Tier-3 硬觸發」的 flag: `--affects-pass-fail-verdict`、`--affects-spec-intent`、`--affects-read-only-file-change`。這一步做的事:確認 Tier 分級不是靠猜的字串,而是由這三個明確的 boolean flag 驅動 `classify_tier()`。

Step 3:實際提出一個 Tier-3(必須詢問人類)問題

```
python -m dv_harness.cli --project-root /tmp/lab9_scratch question-queue add --domain env \
--question "Is the DUT reset polarity active-low per the programming guide?" \
--context-path "dut_facts.clock_reset.resets[0]" \
--option "active-low" --option "active-high" \
--recommendation "active-low" \
--assumption-if-unanswered "Assume active-low per common convention" \
--affects-spec-intent
```

因為帶了 `--affects-spec-intent`, 這一題會命中 `classify_tier()` 的 hard trigger, 一定會落在 Tier 3。真實輸出 (本機實測):

```
{
  "id": "Q-ENV-E7C4D617",
  "question_key": "env::dut_facts.clock_reset.resets[0]:Is the DUT reset polarity active-low per the programming guide?",
  "blocking": true,
  "domain": "env",
  "owner": "DV-owner",
  "tier": 3,
  "tier_reason": "hard_trigger:affects_spec_intent",
  "status": "OPEN",
  "answer": null
}
```

`id` 是 `make_question_id()` 對 `question_key` 做 sha256 再截前 8 碼的結果——只要 domain/context-path/question 三個字串完全一樣, 這個 Q-ID 就會是可重現的固定值 (不是隨機或時間相關), 所以你在自己機器上跑出來的 ID 應該和上面一致。

Step 4: 再提一個「沒有任何 hard trigger」的問題, 觀察它自動落在 Tier-2

```
python -m dv_harness.cli --project-root /tmp/lab9_scratch question-queue add --domain vip \
--question "Which VIP timeout value should this scenario use?" \
--context-path "vip_config.vip_instances[0].timeout" \
--option "1000ns" --option "2000ns" \
--recommendation "1000ns" \
--assumption-if-unanswered "Use 1000ns as a safe low-risk default"
```

真實輸出:

```
{
  "id": "Q-VIP-9C799DA1",
  "tier": 2,
  "tier_reason": "no_hard_trigger_low_blast_radius",
  "status": "ASSUMED",
  "answer": "Use 1000ns as a safe low-risk default",
  "basis": "tier2_safe_to_assume_default (worst case: one wasted, cheaply re-run regression)",
  "decided_by": "dv_harness.question_queue(auto)"
}
```

注意這一題不需要人類介入: `status` 直接變成 `ASSUMED`, `answer` / `basis` / `decided_by` 由 harness 自己填上, 這正是 Part B 規格「Tier-2 = log-and-continue」的行為——它會被記錄進 `decisions.json` / `decisions.md`, 但不會擋住流程繼續往下跑。

Step 5:對 Step 3 的 Tier-3 問題執行真正的人類回答(add/answer round trip 的關鍵一步)

```
python -m dv_harness.cli --project-root /tmp/lab9_scratch question-queue answer Q-ENV-E7C4D617 \
--answer "active-low" --basis "Programming guide section 4.2 states RSTn is active-low" \
--decided-by "lab-student"
```

輸出中 `status` 會從 "OPEN" 變成 "ANSWERED", `answered_at` / `answer` / `basis` / `decided_by` 都會被填上真實值,並且這個決定會被持久化進 `.dv-harness/question_queue/decisions.json` 與可讀的 `decisions.md` ——依照 `QuestionQueueStore` 的設計,同一個 `question_key` 之後再問一次會直接自我解決(Tier-1 `decisions_store_hit`),不會再重新升級給人類。

Step 6:查看整個問題清單,以及 4 項追蹤指標

```
python -m dv_harness.cli --project-root /tmp/lab9_scratch question-queue list
python -m dv_harness.cli --project-root /tmp/lab9_scratch question-queue status
```

`status` 的真實輸出(做完 Step 3-5 之後):

```
{
  "self_resolve_rate_percent": 0.0,
  "self_resolve_rate_target_percent": 90.0,
  "blocking_questions_per_week": 1.0,
  "repeat_question_rate_percent": 0.0,
  "assumption_overturned_rate_percent": 0.0,
  "total_questions": 2,
  "tier1_count": 0,
  "tier2_count": 1,
  "tier3_count": 1,
  "open_blocking_count": 0
}
```

這正是 Part B 要求的 4 項指標: self-resolve rate(對照 90% 目標)、blocking-questions/week、repeat-question-rate、assumption-overturn-rate。 `self_resolve_rate_percent` 是 0,因為我們兩題都不是 Tier-1(沒有任何一題是靠 `manifest_lookup` 自我解決的)——這符合 `module docstring` 自己揭露的殘留限制(見下方常見問題)。

Step 7:用 `python -m dv_harness.loop_telemetry` 檢視 section 107/108 的事件表

`loop_telemetry.py` 沒有掛在 `dv-harness` CLI 底下,只有獨立的 `python -m` 前門:

```
python -m dv_harness.loop_telemetry -h
python -m dv_harness.loop_telemetry names --project-root /tmp/lab9_scratch
python -m dv_harness.loop_telemetry show --project-root /tmp/lab9_scratch
```

`names` 會印出 section 108 的 19 種事件名稱(`LOOP_CREATED`、`LOOP_STARTED` `LOOP_STOPPED`)以及 `loop_state_to_event` 對照表。因為這個 `scratch` 專案從沒真的跑過 `dv-harness start --loop`, `show` 的真實輸出是:

```
(no loop telemetry) NO_LOOP_TELEMETRY_EVENTS: no section-108 loop telemetry event has been written to
.../dv-harness/events.json1. The real producer is engine.DVHarness.loop() -- run
```

```
`dv-harness start --loop "<goal>"` (or Start (loop) on this page); a one-shot `dv-harness run-stage` is not a loop and emits none.
```

exit code 是 2。這是刻意設計的誠實空狀態——`read_loop_telemetry()` 絕不會為了讓畫面好看而捏造一系列假資料。若要在這台機器上真正看到非空的表格,必須先跑一個真正的 `dv-harness start --loop "<goal>"` (那會實際啟動 `engine.DVHarness.loop()`),這超出本 Lab 範圍,本文明確標註:這一步需要真正跑一次 loop,本訓練環境不執行,只示範空狀態下的誠實回報。

Step 8:用 `dv-harness loop-budget` 檢視 section 91-93 的統一 budget engine

```
python -m dv_harness.cli loop-budget --help
python -m dv_harness.cli --project-root /tmp/lab9_scratch loop-budget status
```

`status` 的真實輸出會列出 11 個 budget dimension(`max_iterations`、`max_wall_time`、`max_lsf_jobs`),在一個從未跑過任何 stage 的 scratch 專案上,每一個 `limit` 都是 `null`,但 `source` 欄位會誠實寫出「為什麼沒有被強制」的真實原因,例如:

```
"max_retries": {
  "limit": null,
  "source": "NOT ENFORCED run-wide: policy.max_stage_retries (2) is a PER-GRAPH-NODE attempt budget,
            spent by engine.loop()'s own `ss['attempts'] <= max_retry` test and reset the moment
            current_stage moves on ... Declare loop_budget.limits.max_retries for a real run-scoped
            cap."
}
```

`breaker` 欄位顯示 `{"state": "CLOSED", "opened_at": null, "triggers": []}`——circuit breaker 目前是關閉(正常)狀態。

再示範一次 section 93 的失敗分類引擎(不需要專案根目錄,純文字分類):

```
python -m dv_harness.cli loop-budget classify --text "License checkout failed: no such feature exists (FlexLM)"
```

真實輸出:

```
{
  "failure_type": "LICENSE",
  "retryable": true,
  "rule": "license_checkout_failed",
  "evidence": {"matched_text": "License checkout fail"}
}
```

這示範了 `classify_failure()` 如何從一段真實錯誤文字(模擬 FlexLM 授權失敗訊息)判定出 `FailureType.LICENSE`,並標記為 `retryable: true` (因為授權通常會自然釋放)。

驗證

逐項確認你在自己機器上做完 Step 3-8 之後,能重現以下事實:

- ☐ Step 3 的 Q-ID 是 `Q-ENV-E7C4D617`, `tier` 為 3, `status` 為 "OPEN" (提問當下)。
- ☐ Step 4 的 Q-ID 是 `Q-VIP-9C799DA1`, `tier` 為 2, `status` 一提出就直接是 "ASSUMED",且 `decided_by` 是 `"dv_harness.question_queue(auto)"` (代表是機器自動假設,不是人類)。

- Step 5 執行 `answer` 之後,用 `question-queue list` 再查一次 `Q-ENV-E7C4D617`, `status` 必須已變成 `"ANSWERED"`,且 `answer / basis / decided_by` 都是你剛剛輸入的真實值。
- Step 6 的 `status` 指令中 `total_questions` 為 2、`tier2_count` 為 1、`tier3_count` 為 1。
- Step 7 的 `loop-telemetry show` 在空專案上回傳的 exit code 是 2,且訊息明確點名 `engine.DVHarness.loop()` 是真正的 producer。
- Step 8 的 `loop-budget status` 中,11 個 dimension 全部 `enforced: false`、`limit: null`,但每一個都有非空的 `source` 說明原因(不是靜默留白)。

常見問題

1. 為什麼 `question-queue status` 的 `self_resolve_rate_percent` 一直是 0,離 90% 目標很遠? 這不是 bug,而是 `question_queue.py` 模組自己在 docstring 裡揭露的已知殘留限制:Tier-1 self-resolve 需要一個 `manifest_lookup callable` 被注入(例如接到 `env_manifest.py /MCP client`),但目前沒有任何 **production** 呼叫端會注入這個 **callable**(只有測試會),所以在真實流程上 Tier-1 self-resolve 這條路徑「可用但沒被實際使用」。如果你的專案想拉高 self-resolve rate,必須先確保呼叫 `QuestionQueueStore` 的地方有傳入真正讀取 `env.manifest.json` 的 `manifest_lookup`。
 2. `loop-budget status` 的每個 dimension 都是 `null`,我要怎麼真正讓它「有牙齒」(enforce)? 依照 CLAUDE.md「Unified Loop Budget + Failure Taxonomy + Circuit Breaker」一節的描述,這是刻意的預設值:這個 harness 目前真正在跑的只有 `policy.max_stage_retries` (單一 stage node 的重試上限,跟整個 run 的預算是兩回事)。若要讓某個 dimension 真正被強制,要在專案的 `config.json` 裡宣告 `loop_budget.limits.<dimension_name>` (例如 `loop_budget.limits.max_retries`),否則它永遠停在 NOT ENFORCED、只做「量測但不擋」。
 3. 為什麼 `loop-telemetry show` 一定要先跑過 `dv-harness start --loop` 才有資料, `run-stage` 不行嗎? 因為 section 108 的 19 個事件全都是 `engine.DVHarness.loop()` 這個函式本身在跑迭代迴圈時逐一 emit 出來的(`LOOP_ITERATION_STARTED`、`LOOP_ACTION_SELECTED`.....)。`dv-harness run-stage` 是一次性單跑一個 stage,從未進入 `loop()` 的 `while True` 迭代邏輯,所以結構上就不會產生任何一個事件——這不是資料遺失,而是這個指令本來就不是這個事件表的 producer。CLAUDE.md 對這點也有明文揭露:「a one-shot `dv-harness run-stage` is not a loop and emits none」。
-

Lab 10: Dashboard/GUI 啟動與操作導覽

學習目標

- 能用「真實、已對照原始碼驗證過」的指令啟動 DV Agent Harness 的本地 Dashboard HTTP 伺服器,而不是憑印象猜一個不存在的 CLI 動詞。
- 理解 Dashboard 目前**沒有**專屬的 `dv-harness` CLI 動詞,正確的進入點是 `python -m dv_harness.dashboard`。
- 認識 GUI-19 的 session token 授權機制,能分辨「唯讀 GET」與「會改變狀態的 POST」兩種存取方式的實際差異。
- 走過 Dashboard 上 4 張具代表性的真實卡片(Status、AMBA Fabric / VIP Bind / Scoreboard、Generation Readiness Center、Loop Engineering Center),理解每張卡片背後對應到哪一個真實模組。

前置準備

- 已在 v50 環境內裝好 Python,且 `python -m dv_harness.cli --help` 可以正常跑出完整的子命令清單。
- 準備一個**全新、空的 scratch 目錄**當作這次要觀察的「專案」,不要直接對著 v50 這個 repo 本身啟動 Dashboard——一來避免瀏覽器不小心誤觸某個 POST 動作寫進正式的 `.dv-harness/` 狀態,二來一個乾淨目錄能讓你清楚看到「沒有歷史資料時卡片長什麼樣子」。
- 這整堂課全程屬於 CLAUDE.md Execution Mode Gate 定義的 `LOCAL_ANALYSIS` (純本地啟動一個綁在 `127.0.0.1` 的 HTTP 伺服器,不連任何 Linux/LSF 伺服器),不需要事先做 SSH/Remote Transport 確認。
- **安全提醒**:Dashboard 啟動後是一個**常駐、阻塞(blocking)**的 `ThreadingHTTPServer(...).serve_forever()`,不會自己結束。若你是真人坐在自己的終端機前,直接前景執行、用 `Ctrl+C` 停止即可;若是像本文件產生時一樣,在無人值守的 agent/自動化環境中練習,就必須改用背景執行 + 明確記下 PID + 練習完畢手動 kill 的方式,絕不能留下殘留的背景伺服器行程。以下「步驟」欄位會同時示範這兩種情境。

步驟

步驟 1:確認 Dashboard 沒有專屬的 dv-harness CLI 動詞

很多人第一直覺會猜 `dv-harness dashboard`,但這是錯的——目前 `cli.py` 的子命令清單裡根本沒有這個動詞。用下面指令對照原始碼確認:

```
grep -n '"dashboard"' dv_harness/cli.py
```

做什麼:在 CLI 的 `argparse` 定義檔裡搜尋字面上的 `"dashboard"` 字串。

實際輸出(本 Lab 撰寫時在真實 v50 repo 上執行過):完全沒有輸出——`grep` 找不到任何一行符合,確認 `cli.py` 目前上百個子命令裡真的沒有 `dashboard`。真正的進入點寫在 `dv_harness/dashboard.py` 檔案最尾端:

```
if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser()
    ap.add_argument("--project-root", default=".")
    a = ap.parse_args()
    serve(Path(a.project_root).resolve())
```

也就是說,唯一支援的旗標是 `--project-root` ;沒有 `--port` 可以覆寫。

步驟 2:查真實的預設 host/port(不要用猜的)

Dashboard 監聽的 host/port 不是寫死在 `dashboard.py` 裡,而是來自 `dv_harness/config.py` 的 `DEFAULT_CONFIG["dashboard"]` :

```
python -c "from dv_harness.config import DEFAULT_CONFIG; import json;
          print(json.dumps(DEFAULT_CONFIG['dashboard'], indent=2))"
```

實際輸出:

```
{
  "host": "127.0.0.1",
  "port": 8765,
  "require_auth": true
}
```

這代表:預設只監聽 `127.0.0.1` (不會暴露到區網)、預設 port 是 `8765` ,且 `require_auth: true` ——這是 GUI-19 機制的開關,步驟 4 會實際驗證它的效果。若這個 port 在共用開發機上被佔用,因為沒有 `--port` 可用,唯一辦法是先啟動一次讓 `.dv-harness/config.json` 被自動建立出來,再編輯裡面的 `dashboard.port` 欄位後重新啟動。

步驟 3:在乾淨的 scratch 目錄啟動 Dashboard

```
mkdir -p /tmp/dvh_dashboard_demo
python -u -m dv_harness.dashboard --project-root /tmp/dvh_dashboard_demo
```

(`-u` 關掉 Python 的 stdout 緩衝,確保啟動訊息立即印出來。)

做什麼:啟動一個常駐的 `ThreadingHTTPServer` ,並在接受第一個請求之前,先透過 `dashboard_auth.issue_session_token()` 產生本次 session 專屬的三組角色 token(VIEWER < OPERATOR < APPROVER),刻意避免有任何 mutating 請求趕在 token 存在之前打進來。

實際輸出(本 Lab 撰寫時對一個乾淨 scratch 目錄真的跑出來的結果,每次啟動 token 一定不同):

```
DV Harness Dashboard: http://127.0.0.1:8765/?token=dBGTTXizw1_saGn1xCVND-8N2trZbgAqwlqEOMVopv8
Mutating actions require this session token (also written to .dv-harness/dashboard_session.json).
Roles (VIEWER < OPERATOR < APPROVER) -- the URL above carries APPROVER:
VIEWER: http://127.0.0.1:8765/?token=49q127ipmdh2l2XC29r0q1tOQTMhLka13hhXZICxr4E
OPERATOR: http://127.0.0.1:8765/?token=RhXdlMEJn1Bke_bMa_cBqY9RLSK-9JrVh0X_vH2jeUg
```

若你在自己機器上跑,看到的三組 token 一定跟這裡不一樣——那是正常現象。同一份 token 也會被寫進 `<project-root>/.dv-harness/dashboard_session.json` ,供之後的 API 呼叫核對。若你是在真的終端機操作,就把「APPROVER」那一行完整貼到瀏覽器網址列,會看到本 Lab 開頭提到的那些卡片;若只想看不想操作,也可以不帶 token 直接開 `http://127.0.0.1:8765/` ——唯讀頁面本身不需要 token。

步驟 4:驗證「唯讀 vs. 需要 token」的真實行為

在伺服器還跑著時,另開一個終端機打:

```
curl -s -o /dev/null -w "GET /api/status -> HTTP %{http_code}\n" http://127.0.0.1:8765/api/status
```

實際輸出: GET /api/status -> HTTP 200 ——證實唯讀的 GET 查詢即使完全沒有帶 token 也能正常拿到 200; GUI-19 的 token 檢查只擋在會改變狀態的 POST(例如 APPROVE / TAKEOVER / COSIGN / signoff-export / waiver)上。

步驟 5: 走過 4 張具代表性的真實卡片

用瀏覽器打開步驟 3 給的 URL 後(標題是 <title>DV Agent Harness</title>, 頁首是 <h2>DV Agent Harness L5</h2>), 往下捲可以看到超過 40 張卡片; 下面挑 4 張最貼合本 Lab 主題的來導覽:

1. **Status**(id="tiles", 畫面最上方): 讀 HarnessStatusService (dv_harness/harness_status.py)算出的一句話狀態摘要與明細分頁, 跟 CLI 的 dv-harness status 是同一份底層資料, 兩邊保證不會對不上。
2. **AMBA Fabric / VIP Bind / Scoreboard**(id="ambaFabricCard"): 讀 GET /api/amba, 表格內容就是真實的 AMBA-22 AMBA_PORT_REGISTRY 每一列(Port / Protocol / Fabric Role 等欄位)。卡片自己的說明文字也老實寫著「Discovery and planning only(AMBA-30 / AMBA-31)」——代表這裡只做探索與規劃, 並不會真的觸發任何 bind 動作。
3. **Generation Readiness Center**(id="generationReadinessCard"): 對應 dv_harness/generation_readiness.py 的 section 211 二十列矩陣, 回答「這套 factory 有沒有能力從 spec 產出 subsystem UVM、再從 subsystem 組出 system UVM」, 跟另一張回答「這個專案目前跑到哪一關」的卡片是刻意分開的兩個問題。
4. **Loop Engineering Center**(id="loopCenterCard"): 對應 LOOP_ENGINEERING 整組機制 (loop_contract.py / loop_telemetry.py / loop_convergence.py), 欄位是 Loop | State | Iteration | Verified Gain | Budget | Plateau | Oscillation | Next Action ——目前唯一一個能即時觀察「引擎有沒有卡在原地空轉、有沒有真的在收斂」的面板。

因為 scratch 目錄完全沒有任何 .dv-harness/ 歷史資料, 你會看到大部分卡片顯示「尚無資料」之類的空狀態文字, 而不是假造的示範內容——這正是本專案 Evidence Truth Rule 的體現: 沒有真的跑過的東西, Dashboard 絕不會假裝有。

步驟 6: 安全地停掉伺服器

前景執行的話直接 Ctrl+C。若跟本 Lab 一樣是背景執行, 要用真正監聽該 port 的 PID 去關, 而不是猜一個 shell job id:

```
netstat -ano | grep '127.0.0.1:8765' | grep LISTENING
# 取最後一欄的 PID 之後
taskkill //PID <PID> //F           # Windows
kill <PID>                         # 若在真正的 Linux/macOS 終端機
```

實際輸出(本 Lab 撰寫時的真實結果): 關閉前 netstat 顯示 TCP 127.0.0.1:8765 ... LISTENING; taskkill 成功後幾秒內再查, 同一個 port 只剩 TIME_WAIT (甚至完全查不到), 確認伺服器行程真的結束, 沒有留下背景殘留。

驗證

- ☐ grep -n '"dashboard"' dv_harness/cli.py 沒有任何輸出, 確認自己沒有誤用不存在的 CLI 動詞。
- ☐ 啟動指令印出的第一行是 DV Harness Dashboard: http://127.0.0.1:<port>/?token=..., 且帶著 VIEWER / OPERATOR / APPROVER 三組不同 token。

- 瀏覽器(或 `curl`)打開該網址,能看到標題為「DV Agent Harness」的頁面,往下能捲到 Status、AMBA Fabric / VIP Bind / Scoreboard、Generation Readiness Center、Loop Engineering Center 這四張卡片。
- `curl -o /dev/null -w '%{http_code}' http://127.0.0.1:<port>/api/status` 回傳 `200`,證實唯讀查詢不需要帶 token。
- 練習結束後, `netstat -ano | grep <port>` 已看不到 `LISTENING` 狀態。

常見問題

1. 啟動後瀏覽器連線被拒絕:先確認沒有跑錯指令——這裡沒有 `dv-harness dashboard` 這個動詞(步驟 1 已驗證),只有 `python -m dv_harness.dashboard --project-root <dir>`。另外 Dashboard 預設只監聽 `127.0.0.1 ( DEFAULT_CONFIG["dashboard"]["host"] )`,若想從另一台機器連進來預設連不到,得自行去 `.dv-harness/config.json` 改 `dashboard.host`。
2. **8765 port 已被佔用**:目前唯一的命令列參數是 `--project-root`,沒有 `--port` 可覆寫。要換 port 只能先跑一次讓 `.dv-harness/config.json` 自動被建立,再手動改裡面的 `dashboard.port` 後重跑——這也是為什麼在多人共用開發機上, `USAGE_MULTI_USER_SAFETY.md` 特別強調「never share a `--project-root`」,每人本來就該指向不同專案、自然也就用不同 port。
3. **點某個 POST 動作(例如 APPROVE / TAKEOVER)一直被 403 擋掉**:這是 GUI-19 機制刻意設計的行為,不是 bug——`require_auth: true` (預設值)時,所有會改變狀態的 POST 都必須帶正確的 session token,且必須是對應角色(VIEWER < OPERATOR < APPROVER)。步驟 3 印出的三組 URL 分別對應三種角色,token 貼錯角色一樣會被拒絕。若真的要在完全信任的單機環境暫時關掉這道防線, `dashboard_auth.startup_banner()` 在 `require_auth: false` 時會明確印出警告文字「every mutating POST ... is open to any local process」——這是文件裡白紙黑字承認的已揭露限制,正式環境不建議關掉。
4. 偶爾看到 Dashboard 讀取 `state.json` 短暫失敗(Windows 特有): `dashboard.py` 的 `_read_json_file()` 自己的原始碼註解就寫明這是已知、已處理過的 Windows 特有情況——背景執行緒正在用 `os.replace()` 原子覆寫 `state.json` 的同一瞬間,另一支 HTTP 讀取執行緒可能拿到 `WinError 5 (PermissionError)`,程式碼會自動重試最多 10 次,不會直接把 500 丟回前端。若自己在寫類似的整合程式,也要記得處理這個已揭露的 race window,而不是假設檔案永遠一次讀成功。

相關檔案路徑(供後續查閱):

- `D:\DV\Task\DV_Agent_Harness_L5\v50\dv_harness\dashboard.py`
 - `D:\DV\Task\DV_Agent_Harness_L5\v50\dv_harness\dashboard_auth.py`
 - `D:\DV\Task\DV_Agent_Harness_L5\v50\dv_harness\config.py`
 - `D:\DV\Task\DV_Agent_Harness_L5\v50\dv_harness\cli.py`
-

Lab 11: 疑難排解與下一步學習資源

學習目標

- 能夠正確判讀 `dv-harness` CLI 常見的錯誤訊息與結束碼(exit code),而不是看到非 0 就恐慌重跑。
- 理解 Harness 特有的「拒答式失敗」(fail-closed)設計:找不到必要條件時回報明確原因,而不是猜測或捏造結果,這是 Evidence Truth Rule 的具體展現。
- 認識 `gate_verdict` (如 `MISSING_EVIDENCE`、`GATE_FAIL`)等跨模組共用的判定詞彙,並知道它們分別代表什麼處置動作。
- 知道在沒有真實 LSF/License Server 連線的訓練環境中,`dv-harness status` 會如何誠實回報「探測不到」而非假裝一切正常,並學會下一步該去哪裡查更完整的文件。

前置準備

- 已完成前面幾個 Lab,熟悉在 `D:\DV\Task\DV_Agent_Harness_L5\v50` 底下以 `python -m dv_harness.cli` `<verb> --help` 查詢子命令的習慣。
- 手邊有一個可自由讀寫的暫存目錄,本 Lab 一律使用範例路徑 `/tmp/dvlab11_scratch` (Windows 環境下對應到你自己的暫存資料夾即可),絕不對 `v50` 專案本體目錄下指令。
- 已閱讀過 `CLAUDE.md` 開頭的「Evidence Truth Rule」段落(第 20 行起),了解「證據為王、CLAUDE.md 只是指引不是證據」的原則——本 Lab 看到的每一個錯誤訊息,本質上都是這條規則的落地實作。

步驟

Step 1:重新確認 CLI 的真實 verb 清單,作為排錯的起點

```
python -m dv_harness.cli --help
```

這個指令印出目前 CLI 真正支援的所有一級 verb(`start`、`status`、`run-stage`、`reference-audit`、`git-guard`、`coverage-db-integrity`上百個)。疑難排解的第一步永遠是回到這裡確認你打的 verb 真的存在,而不是憑記憶或舊文件猜測。

Step 2:故意打錯 verb,觀察真實的「未知子命令」錯誤

```
python -m dv_harness.cli notaverb
echo "exit code: $?"
```

實測輸出是完整的 usage 清單,最後一行:

```
dv-harness: error: argument cmd: invalid choice: 'notaverb' (choose from start, run-stage, status, ...)
```

結束碼是 `2` (argparse 的標準用法錯誤碼)。重點:這個 `2` 是 argparse 本身的通用錯誤碼,跟後面 Step 6 會看到的、Harness 自訂的業務邏輯錯誤碼(同樣可能是 2,但意義完全不同)不要混為一談——一定要看訊息內容,不能只看數字。

Step 3:一個真實會踩到的坑——全域旗標放錯位置

`--project-root` 是全域旗標,必須寫在 `verb` 之前;如果誤放在 `verb` 之後,`argparse` 會把它當成該子命令自己的位置參數去解析,產生一個看起來莫名其妙的錯誤:

```
python -m dv_harness.cli status --project-root /tmp/does_not_exist_xyz
```

實測輸出:

```
dv-harness status: error: argument status_view: invalid choice: 'C:/Users/.../does_not_exist_xyz'
(choose from full, blockers, jobs, coverage, agents, system, evidence, signoff)
```

正確寫法是把 `--project-root` 移到 `verb` 前面:

```
python -m dv_harness.cli --project-root /tmp/dvlab11_scratch status
```

Step 4:在乾淨的 `scratch` 專案上跑 `status`,讀懂「誠實降級」的輸出

```
mkdir -p /tmp/dvlab11_scratch
python -m dv_harness.cli --project-root /tmp/dvlab11_scratch status
```

真實輸出的 JSON 裡有一段 `degraded_probe_transport`:

```
"degraded_probe_transport": {
  "requested": "auto",
  "resolved": "none",
  "available": false,
  "reason": "auto: no usable probe transport here -- relay: RELAY_NOT_READY; local:
    LOCAL_COMMANDS_MISSING: lmutil, bqueues. Resource probing stays OFF rather than reading a
    missing binary as a full license or a jammed farm."
}
```

這正是本專案「找不到證據就誠實說找不到,絕不瞎猜」的活教材:本訓練環境沒有真正的 License Server(`lmutil`)也沒有真正的 LSF 佇列(`bqueues`),Harness 不會把「探測不到」誤判成「授權滿了」或「farm 卡住了」,而是把 `available` 設為 `false` 並把缺什麼講清楚。

Step 5:用 `explain` 看懂 `gate_verdict` 詞彙表

```
python -m dv_harness.cli explain --stage ENV_CHECK
```

在真實專案根目錄(不加 `--project-root`)下執行,輸出的「Current run state」區塊會出現:

```
"gate_verdict": "MISSING_EVIDENCE",
"gate_reasons": ["execution_mode_validator: no evidence block supplied"]
```

`dv_harness/gates.py` 的 `evaluate_stage_evidence()` docstring 明確定義了這一組跨所有 stage 共用的判定詞彙,之後你在任何 Lab 遇到卡關都可以回來對照:

verdict	意義
NO_GATE_REQUIRED	該 stage 沒有掛 gate,視同直接通過
PASS	所有掛的 gate 都跑過且自己的檢查都通過

verdict	意義
MISSING_EVIDENCE	該 stage 有掛 gate,但 agent 的回覆裡完全沒放對應的證據區塊
NEEDS_USER_INPUT	gate 失敗但失敗的原因是「需要人類回答」,不是 agent 做錯
DV_REVIEW_PENDING	腳本檢查都過,但有 DV_JUDGMENT 欄位還沒被人類會簽
GATE_FAIL	真的有 gate 自己的腳本檢查沒過

只有 NO_GATE_REQUIRED / PASS 會把 stage 推進到 Status.PASS,其餘一律被 engine.run_stage 降級為 PARTIAL (NEEDS_USER_INPUT 例外,會變成 WAIT_USER)。

Step 6:觸發一個真實的業務層 JSON 錯誤,並讀懂它的結束碼

```
python -m dv_harness.cli coverage-db-integrity check --merge-request /tmp/dvlab11_scratch/nope.json
echo "exit code: $?"
```

真實輸出:

```
{
  "error": "MERGE_REQUEST_UNREADABLE",
  "detail": {
    "path": "...\\nope.json",
    "error": "[Errno 2] No such file or directory: '...\\nope.json'"
  }
}
```

結束碼是 2。注意這個 verb 走的是「頂層一定有 error 欄位、detail 帶原始例外」的固定 JSON 錯誤慣例——很多 Harness 子命令都遵循同一套慣例,養成先看 error 欄位字串(而不是純看 exit code 數字)的習慣,才能真正定位問題。

Step 7:在沒有真實 push 內容時,看懂「允許但誠實標記」的輸出

```
python -m dv_harness.cli --project-root /tmp/dvlab11_scratch git-guard --check pre-push < /dev/null
```

由於 /tmp/dvlab11_scratch 不是 git repo、也沒有真正要推送的 commit,真實輸出是:

```
{
  "hook": "pre-push",
  "allowed": true,
  "reason": "no protected branch (main/master) among the pushed refs",
  "blast_radius": {
    "tier": "NOT_ASSESSABLE",
    "reasons": ["no real diff to measure (status=NO_PUSHED_COMMITS); allowing, and saying so rather than inventing a tier"]
  }
}
```

這也是同一套原則的另一個表現:量不到真實 diff 時回報 NOT_ASSESSABLE 並解釋原因,而不是硬套一個等級。

驗證

- ☐ 能重現 Step 2 的 invalid choice 錯誤,且能說出結束碼 2 是 argparse 的通用錯誤碼。

- 能解釋 Step 3 為什麼 `--project-root` 放在 verb 之後會被誤解成子命令自己的位置參數。
- 能在自己的 scratch 目錄重現 Step 4 的 `LOCAL_COMMANDS_MISSING: lmutil, bqueues`, 並說明這代表「本環境沒有真正的 LSF/License 工具, Harness 選擇誠實降級而非瞎猜」。
- 能默寫出 Step 5 表格中至少 4 個 `gate_verdict` 值分別對應的意義。
- 能重現 Step 6 的 `MERGE_REQUEST_UNREADABLE` 錯誤 JSON, 並指出 `error` 欄位才是判斷根因的第一手資訊。

常見問題

1. Q: `status` 印出 `"degraded": false` 但 `degraded_probe_transport.available` 卻是 `false`, 是不是矛盾? A: 不矛盾。`degraded` (全域降級旗標) 跟單一 `probe transport` 探測結果是分開追蹤的兩件事; 探測不到 `lmutil / bqueues` 只代表「這次沒有資源探測數據可用」, 不會自動把整個系統判定成降級狀態——這正是 CLAUDE.md 反覆強調的「不要把兩種不同的『不知道』混為一談」的具體案例。
2. Q: 同一個「非 0」結束碼在不同 verb 裡意義差很多, 要怎麼確定? A: 每個 verb 的結束碼含意都不是全域統一的, 必須以該 verb 自己的 `--help` 與 CLAUDE.md 裡對應的段落(用關鍵字如 `exit codes` 搜尋該 verb 名稱)為準; 本 Lab 示範的 `coverage-db-integrity check` 用 `2` 代表輸入檔讀不到, 但其他 verb (如 CLAUDE.md 中記載的一些 gate 類 verb) 用 `2 ~ 6` 分別代表完全不同的失敗類型, 切勿套用「0 成功、非 0 失敗」這種過度簡化的假設。
3. Q: 這個 Lab 只教了幾個 verb, 其他上百個 verb 遇到問題要去哪裡查? A: 兩個入口都要用: (1) 對任何 verb 先跑 `python -m dv_harness.cli <verb> --help` 拿到即時、保證與程式碼同步的旗標與用法; (2) 到 `docs/DV_Agent_Harness_L5_Detailed_User_Guide_TC_2026-09-06.md` (全文約 2677 行) 查第 16 章「完整 CLI 指令參考與端到端使用範例」, 以及第 9 章「模擬、除錯與 LSF Regression」查跟本 Lab 相關的除錯情境; 如果是想確認某個行為是不是專案的既定設計(而非 bug), 回到專案根目錄的 `CLAUDE.md` (全文約 10127 行) 用關鍵字搜尋該 verb 名稱或錯誤字串(例如搜尋 `MISSING_EVIDENCE` 或 `LOCAL_COMMANDS_MISSING`), 幾乎都能找到當初設計這個行為的完整理由段落。任何時候 `CLAUDE.md` 的文字描述與你眼前指令的真實輸出對不上, 一律以真實輸出(現場證據)為準, 這就是 Evidence Truth Rule 的最後一道防線。